



STUDENT REPORTING LOGBOOK

BIT3208: Advanced Web Design and Development

Student Details

Student Name: MARTIN MIGWI MUNYI

Admission Number: BBIT/2024/74296

Course/Class: BACHELOR OF BUSINESS INFORMATION TECHNOLOGY

Unit Code: BIT3208

Unit Name: Advanced Web Design and Development

Lecturer Name: MR. NYORO MICHAEL

Semester/Academic Year: MAY – AUGUST 2026

Project Details

Project Title: Student Management System

Selected Technologies: PHP for server-side scripting and business logic

MySQL for database management

HTML5, CSS3, and JavaScript for frontend development

Responsive design using Flexbox and CSS Grid

GitHub for version control

GitHub Repository Link: https://github.com/Martin254-gif/BIT3208_Project

TABLE OF CONTENTS:

Week 1: Local Environment Setup

1.1 Installation of XAMPP/WAMP/Laragon	8
1.2 Apache and MySQL Configuration	9
1.3 Localhost Testing	10
1.4 Hello World Test	10
1.5 Database Connection Test	11
1.6 Student Reflection	11

Week 2: Wireframes and GUI Design

2.1 login page and homepage Wireframes	12
2.2 Dashboard Layout Design	13
2.3 Mobile View Mockup	14
2.4 Colour Theme and Navigation Structure	15
2.5 GUI Prototype in Figma/Canva/Draw.io	15
2.6 Student Reflection	15

Week 3: JavaScript and PHP Basics

3.1 JavaScript Form Validation	16
3.2 Password Strength Checker	17
3.3 PHP Syntax Practice	19
3.4 Database Connection Script	20
3.5 Dynamic User Input Handling	21
3.6 Student Reflection	21

Week 4: Dynamic Backend Processing

4.1 Login Form Implementation	22
4.2 Registration Form Implementation	23
4.3 Contact Form Implementation	24
4.4 Form Validation System	25
4.5 PHP Processing Pages	26
4.6 Folder Organisation	27
4.7 Database Connection Integration	28
4.8 Student Reflection	28

Week 5: Database and CRUD Operations

5.1 Database Creation	29
5.2 Table Creation	30
5.3 Add Student (Create Operation)	31
5.4 View Students (Read Operation)	32
5.5 Edit Student (Update Operation)	33
5.6 Delete Student (Delete Operation)	34
5.7 Database Connection Script	35
5.8 CRUD Complete System Overview	36
5.9 Student Reflection	37

Week 6: Database Integration and CRUD Operations

6.1 Dashboard with Statistics	37
6.2 Student List with Search Functionality	38
6.3 Add Student Enhancement	39
6.4 Edit Student Enhancement	40
6.5 Delete Confirmation Enhancement	41
6.6 Search Functionality Implementation	42
6.7 Database Structure Review	43
6.8 Student Reflection	43

Week 7: User Authentication and Sessions

7.1 Registration System	44
7.2 Login System	45
7.3 Dashboard with Role-Based Access	46
7.4 User Profile Management	47
7.5 Logout Functionality	48
7.6 Users Table Structure	49
7.7 Protected Pages Implementation	50
7.8 Student Reflection	50

Week 8: Responsive Web Design

8.1 Desktop View Design	51
8.2 Tablet View Design	52
8.3 Mobile View Design	53
8.4 Responsive Navigation	54
8.5 Responsive Tables	55
8.6 Responsive Forms	56
8.7 Mobile Menu Implementation	57
8.8 Student Reflection	58
8.9 Challenges Encountered	58
9.0 Conclusion	58

Week 1: Local Environment Setup

Title : Installation and Testing of Local Development Environment

Fig 1 Installation of XAMPP/WAMP/Laragon



Fig 2: Apache and MySQL Running

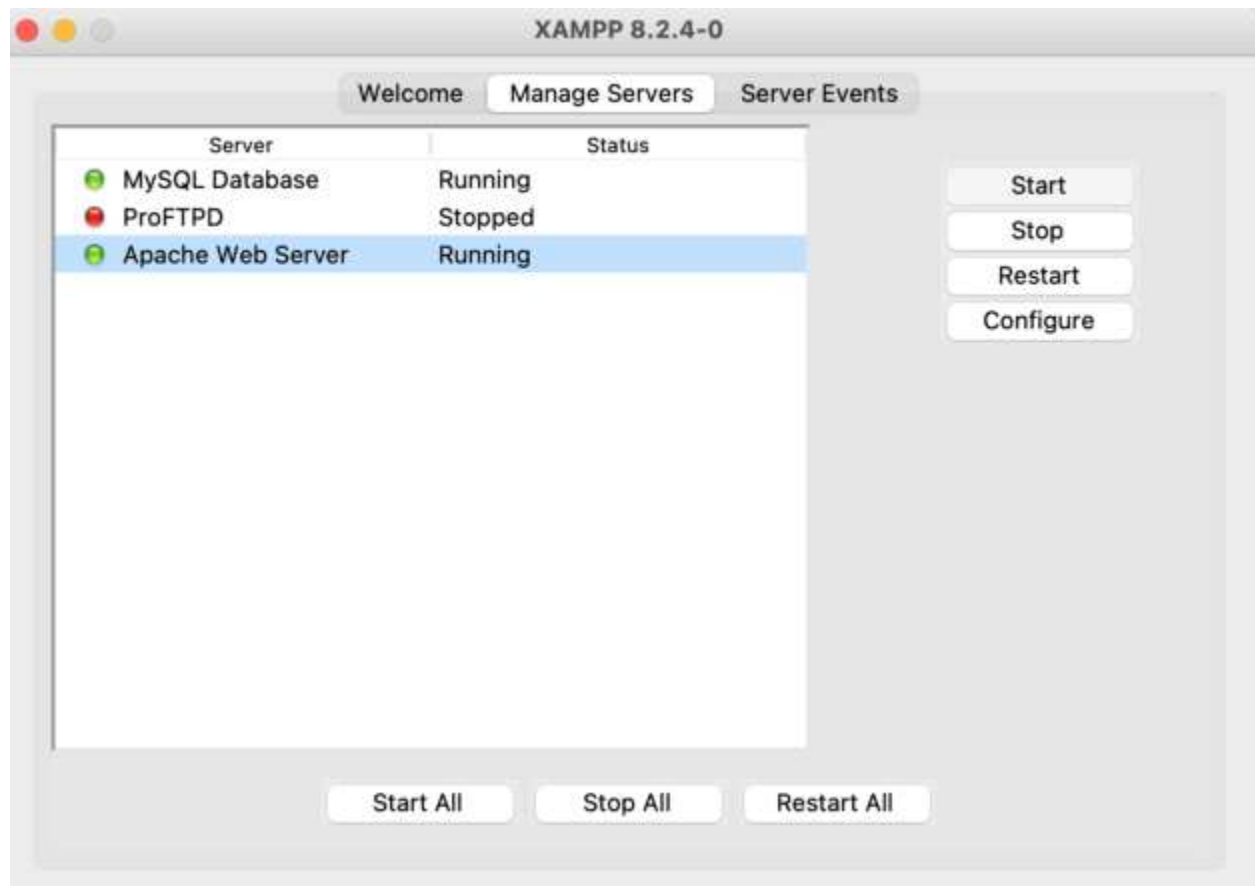


Fig 3: Localhost Test Page



Fig 4: Hello World Test



Fig 5: Database Connection Test



Student Reflection:

During Week 1, I successfully installed XAMPP on macOS and configured the local development environment. The XAMPP Control Panel shows Apache and MySQL services running properly. Fig 1 demonstrates the successful installation of XAMPP v8.2.4. Fig 2 confirms both Apache and MySQL are active and listening on their respective ports. Fig 3 shows the localhost test page rendering correctly, confirming the web server is operational. Fig 4 displays a simple PHP "Hello World" page, proving PHP processing is functional. Fig 5 shows successful database connection using `mysqli_connect()`, confirming PHP and MySQL are properly integrated. The main challenge was configuring the XAMPP permissions on macOS, which was resolved by adjusting folder permissions using terminal commands.

Week 2: Wireframes and GUI Design

Title : User Interface Planning and System Design

Fig 1: Login page and home page Wireframe

The image shows a wireframe for a web application, divided into two main sections: a login page and a home page.

Login Page:

- Header: "Sign in to your account" with a key icon.
- Form fields: "Username" (placeholder: "Enter your username") and "Password" (placeholder: "Enter your password").
- Checkbox: "Remember me".
- Button: "SIGN IN".
- Text: "Demo: admin / admin123".

Home Page:

- Header: "Student Management" (left) and "Admin" (right) with a bell icon.
- Left Sidebar: "Dashboard", "Students", and "Add Student".
- Main Content: "Welcome back, Admin!"
- Summary Cards: "STUDENTS 150", "COURSES 12", and "NEW 5".
- Table:

Student Name	Course	Status
John Doe	Math	Active
Jane Smith	Science	Active

Logout

Fig 2: Dashboard Layout Design

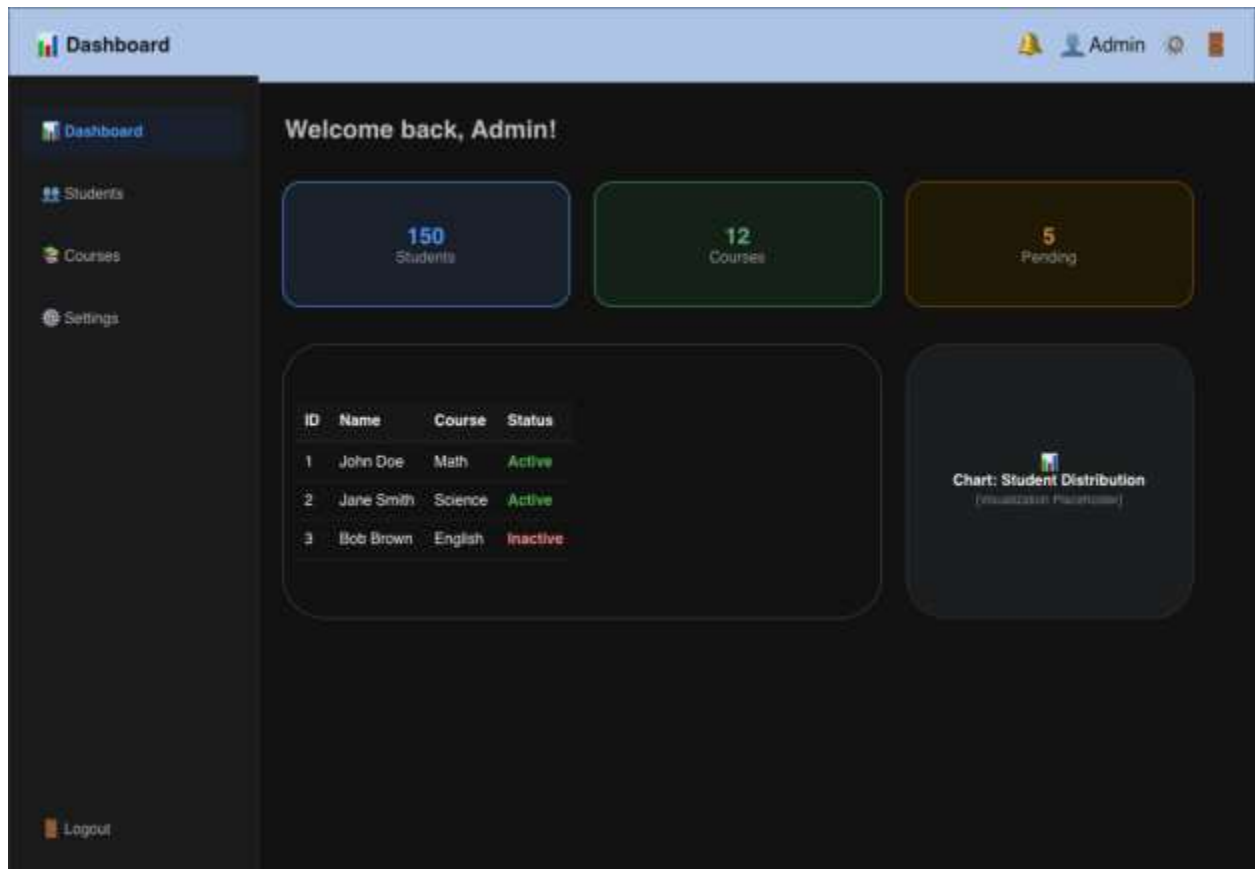


Fig 3: Mobile View Mockup



Fig 4: Colour Theme and Navigation Structure

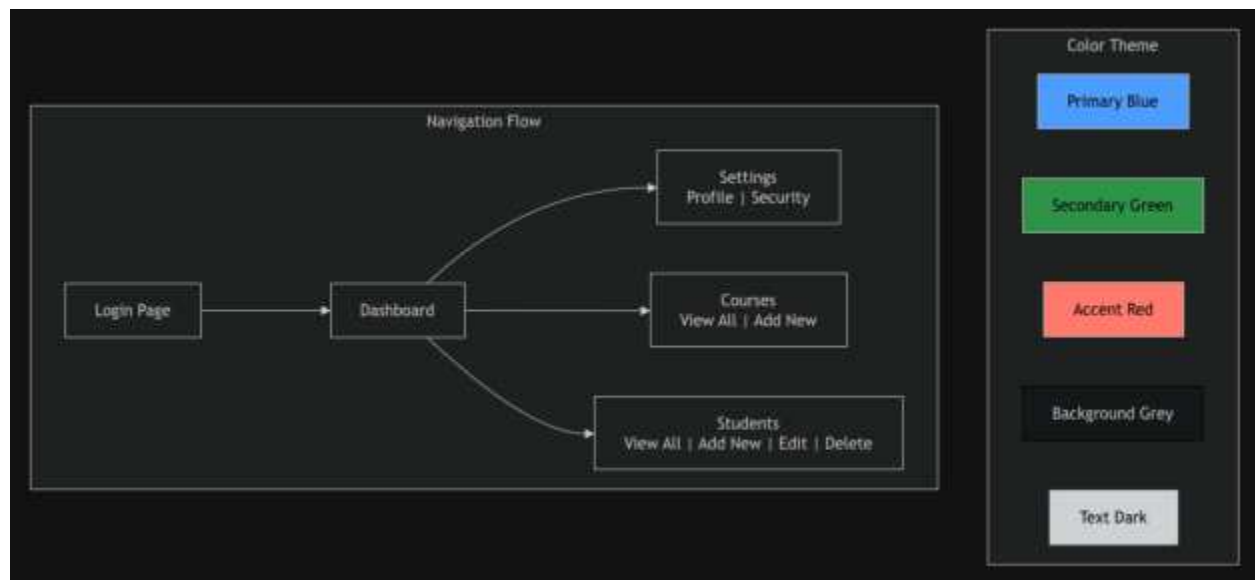
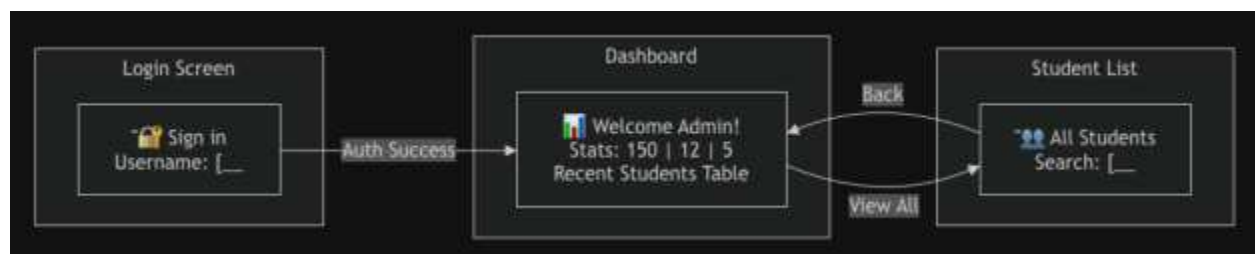


Fig 5: GUI Prototype in Figma/Canva/Draw.io



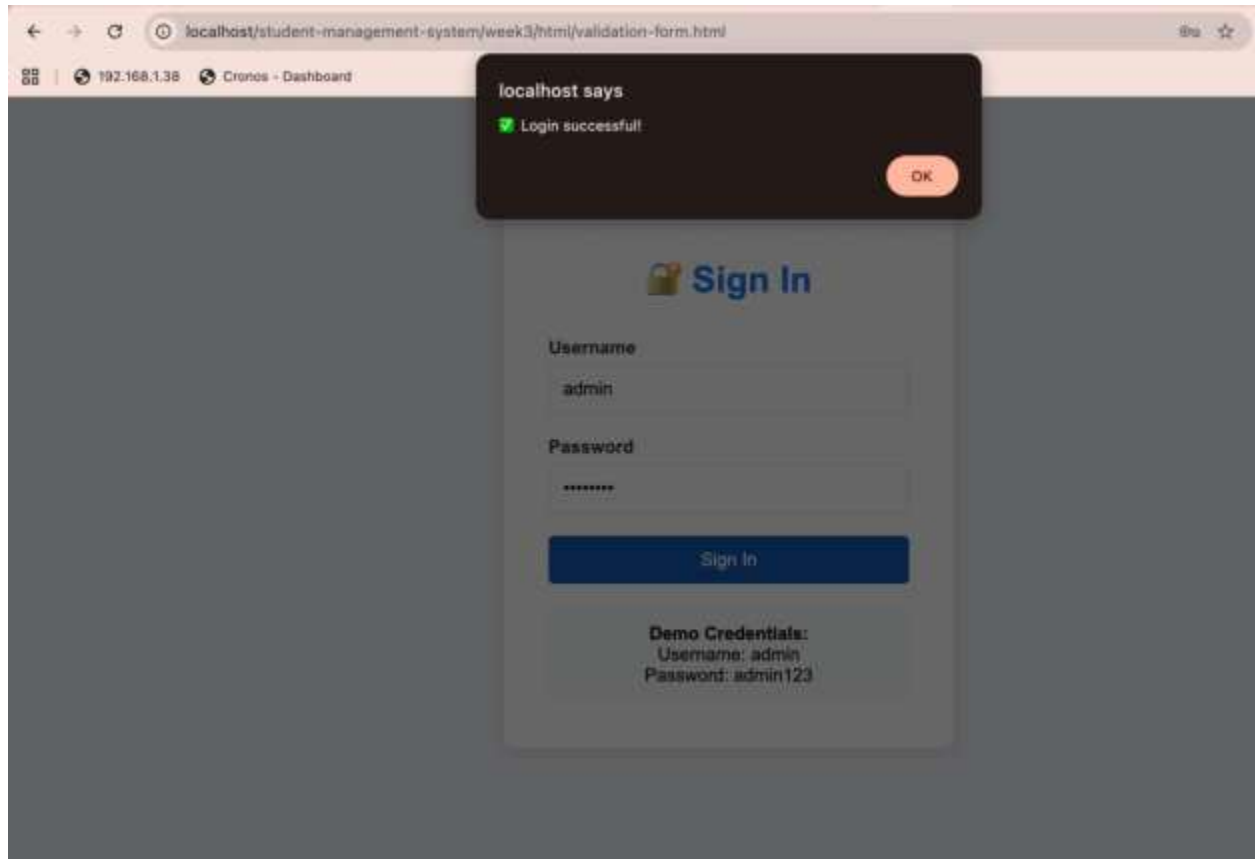
Student Reflection

The dashboard layout was designed to reduce user clicks and improve accessibility. The navigation menu was positioned on the left for easier scalability as more modules are added (students, courses, reports, settings). The colour theme (#1a73e8 primary blue) was selected for its professional appearance and association with trust in educational software. User experience considerations included clear visual hierarchy, consistent button styles, and responsive behaviour across devices. The mobile mockup ensures the system remains usable on smartphones, which is critical as students and lecturers increasingly access systems from mobile devices. The intended users are school administrators, lecturers, and students, each requiring different permission levels. Draw.io was used to create the prototype due to its free availability and compatibility with the course requirements.

WEEK 3: JavaScript and PHP Basics

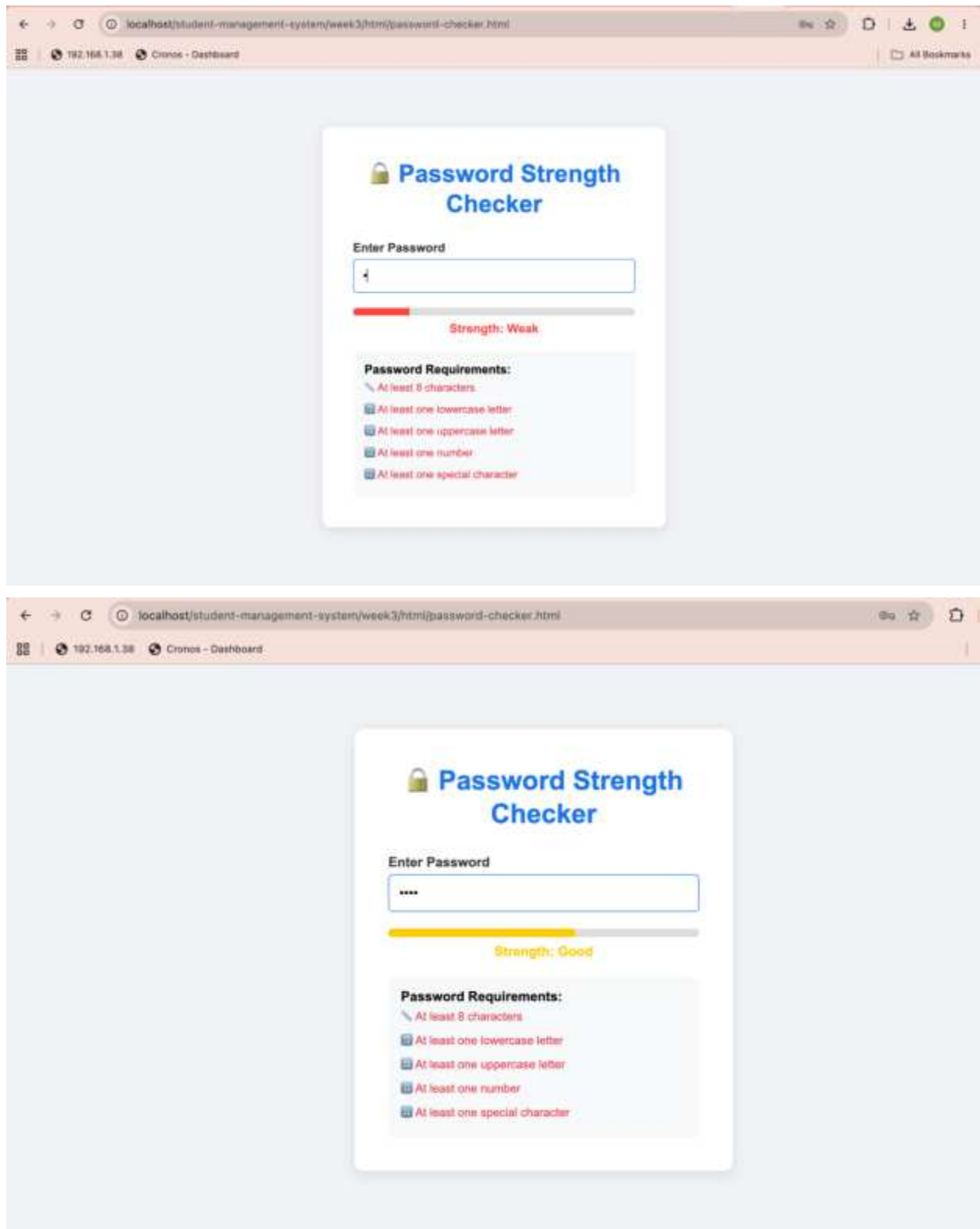
Title : Frontend Interaction and Backend Foundations

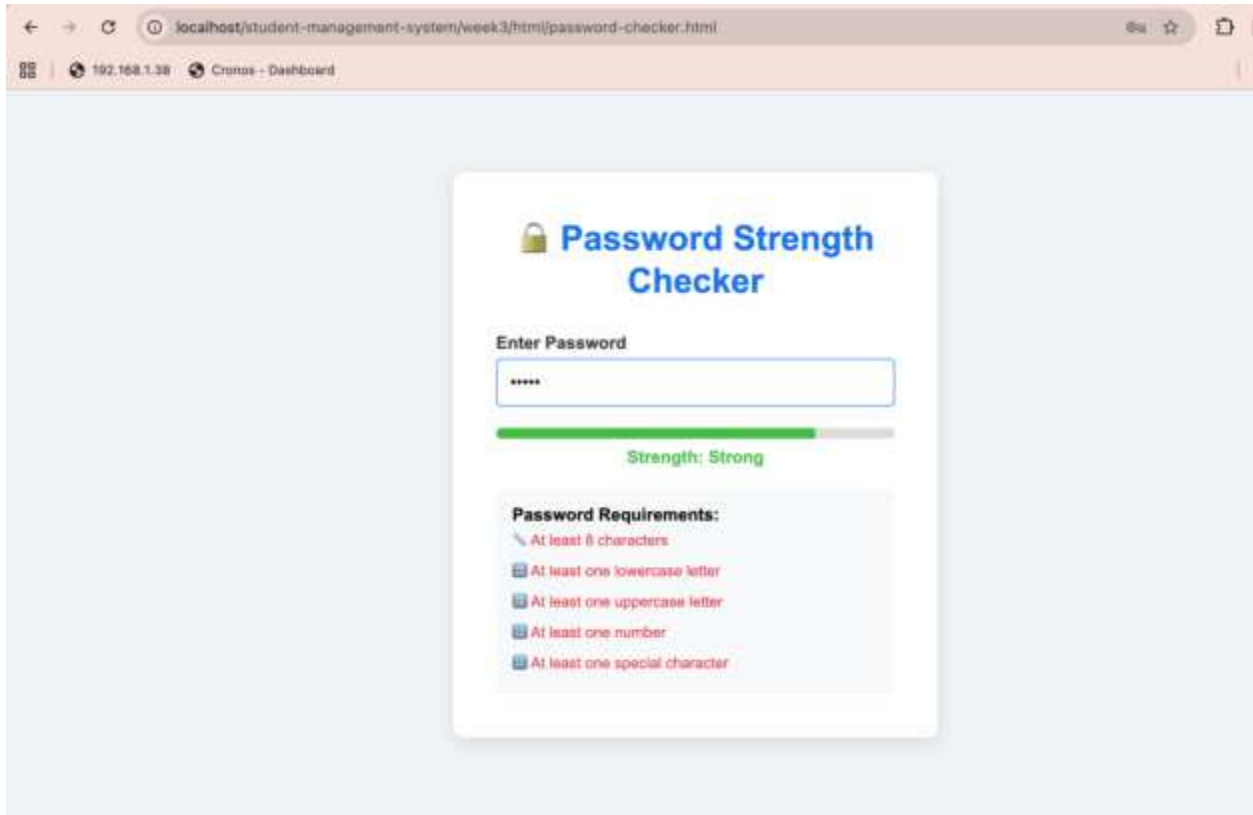
Fig 1: JavaScript Form Validation



JavaScript form validation showing error messages for empty fields. The username field is highlighted with a red border when validation fails. The validation ensures users cannot submit empty forms, providing immediate feedback before data is sent to the server.

Fig 2: Password Strength Checker





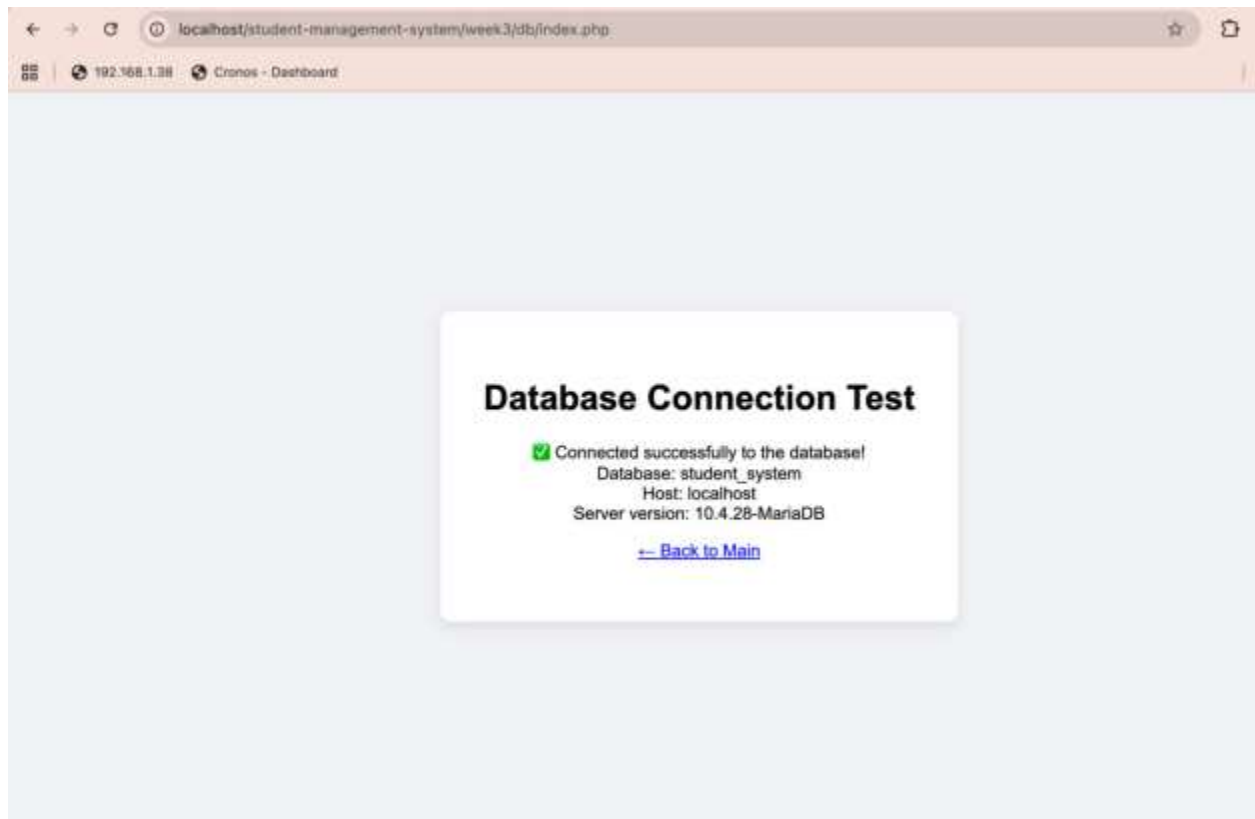
Password strength checker providing real-time feedback as users type. The password strength is displayed with a colored bar and text indicator (Weak, Fair, Good, Strong, Very Strong). This improves security by encouraging users to create stronger passwords.

Fig 3: PHP Syntax Practice



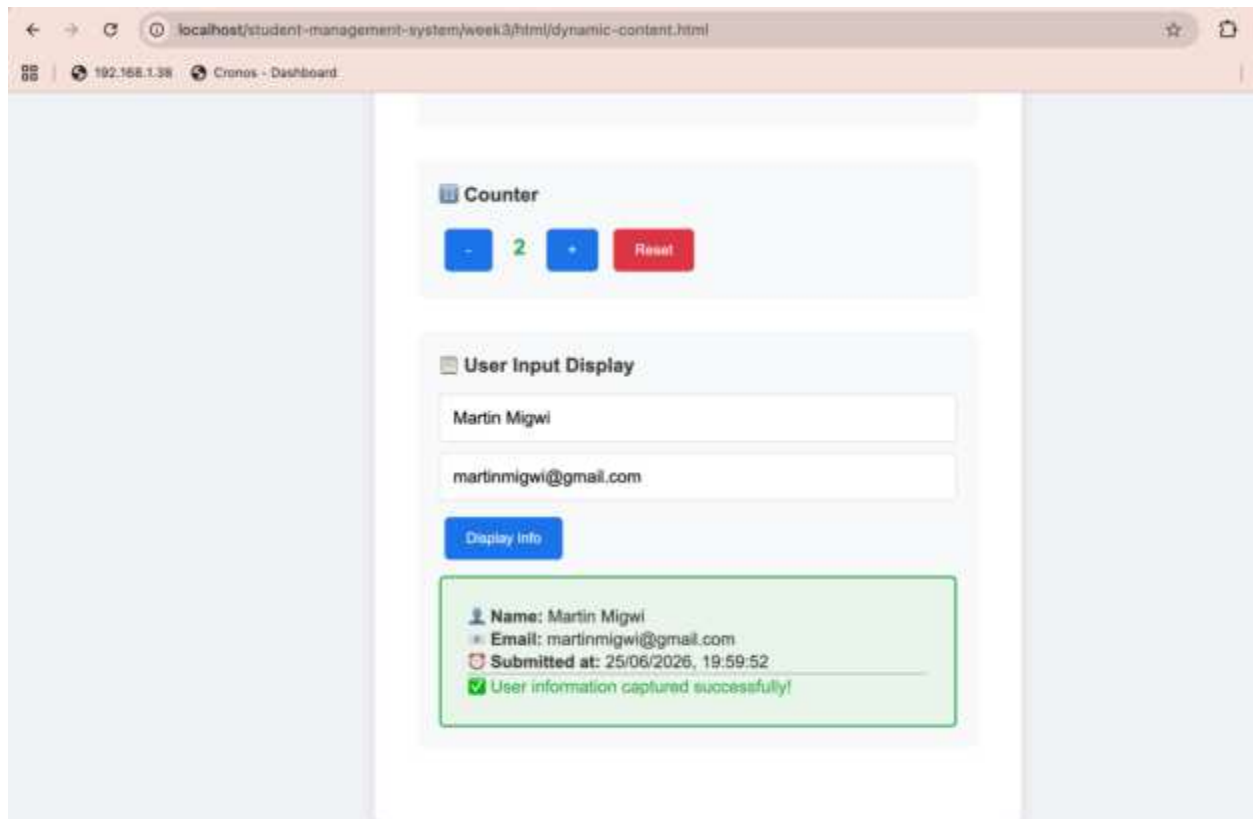
PHP syntax practice page demonstrating basic PHP programming concepts including variable declaration, array operations, conditional statements, loops, and custom functions. This page was used to practice server-side scripting fundamentals.

Fig 4: Database Connection Script



Database connection script successfully connecting to the student_system database. The `mysqli_connect()` function establishes the connection, confirming successful connection by displaying server information and database name.

Fig 5: Dynamic User Input Handling



Dynamic user input handling demonstrating DOM manipulation. JavaScript captures user input in real-time, displays it dynamically on the page, and provides interactive features such as live text preview, counter, and colour changer.

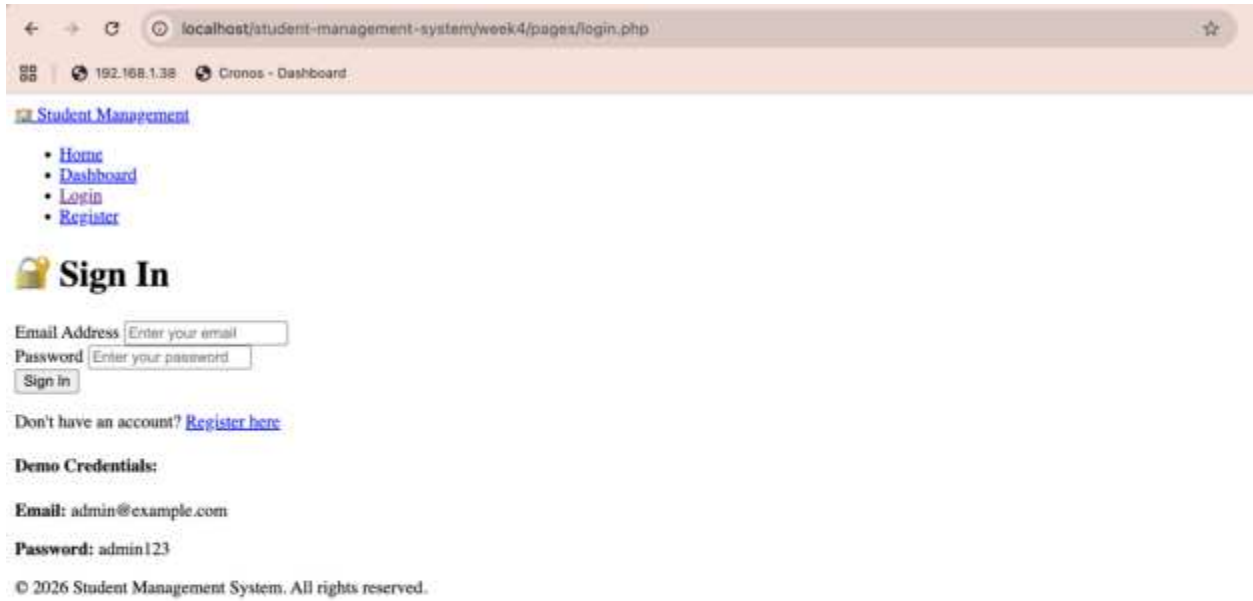
Student Reflection

The login validation prevents users from submitting empty forms and validates email format before data reaches the server. This improves user experience and reduces server load. PHP was used to process form data through `$_POST`, while MySQL stored the records. JavaScript improved user interaction before data reaches the server. The password strength checker encourages users to create stronger passwords by providing real-time feedback, improving overall system security. The database connection script demonstrates the foundation for all database operations - without a successful connection, the system would be unable to store or retrieve student records. Technologies used include HTML, CSS, JavaScript for frontend, PHP for backend, and MySQL for database.

WEEK 4: Dynamic Backend Processing

Title : Dynamic Backend Processing and Form Integration

Fig 1: Login Form Screenshot



The screenshot shows a web browser window with the address bar displaying 'localhost/student-management-system/week4/pages/login.php'. The browser's address bar also shows the IP address '192.168.1.38' and the page title 'Cronos - Dashboard'. The page content includes a navigation menu with links for 'Home', 'Dashboard', 'Login', and 'Register'. Below the menu is a 'Sign In' section with a lock icon. It contains two input fields: 'Email Address' with a placeholder 'Enter your email' and 'Password' with a placeholder 'Enter your password'. A 'Sign in' button is located below the password field. A link 'Register here' is provided for users who don't have an account. Below the sign-in form, 'Demo Credentials' are listed: 'Email: admin@example.com' and 'Password: admin123'. At the bottom, a copyright notice reads '© 2026 Student Management System. All rights reserved.'

Student Management

- [Home](#)
- [Dashboard](#)
- [Login](#)
- [Register](#)

 **Sign In**

Email Address

Password

Don't have an account? [Register here](#)

Demo Credentials:

Email: admin@example.com

Password: admin123

© 2026 Student Management System. All rights reserved.

Login form interface with email and password fields. The form includes validation messages and demo credentials for testing purposes.

Fig 2: Registration Form Screenshot

The screenshot shows a web browser window with the address bar displaying 'localhost/student-management-system/week4/pages/register.php'. The browser's address bar also shows the IP address '192.168.1.38' and the page title 'Cronos - Dashboard'. The page content includes a navigation menu with links for 'Home', 'Dashboard', 'Login', and 'Register'. Below the menu is a section titled 'Create an Account' with a pencil icon. The registration form contains four input fields: 'Full Name' (filled with 'Martin Migwi'), 'Email Address' (filled with 'martin@gmail.com'), 'Password (min 6 characters)' (filled with '*****'), and 'Confirm Password' (filled with '*****'). A 'Register' button is located below the form. Below the form, there is a link 'Already have an account? Login here'. At the bottom of the page, a copyright notice reads '© 2026 Student Management System. All rights reserved.'

Student Management

- [Home](#)
- [Dashboard](#)
- [Login](#)
- [Register](#)

Create an Account

Full Name

Email Address

Password (min 6 characters)

Confirm Password

Already have an account? [Login here](#)

© 2026 Student Management System. All rights reserved.

Registration form with full name, email, password, and confirm password fields. The form includes client-side and server-side validation to ensure data integrity. Password hashing using `password_hash()` is implemented for security.

Fig 3: Contact Form Screenshot

The screenshot shows a web browser window with the address bar displaying 'localhost/student-management-system/week4/pages/contact.php'. The browser's address bar also shows the IP address '192.168.1.38' and the page title 'Cronos - Dashboard'. The page content includes a navigation menu with links for 'Home', 'Dashboard', 'Login', and 'Register'. Below the navigation menu is a section titled 'Contact Us'. The contact form contains four input fields: 'Full Name' (filled with 'Martin Migwi'), 'Email Address' (filled with 'martin@gmail.com'), 'Subject' (filled with 'Missing exam marks'), and 'Message' (filled with 'Exams were recently released, but some of my marks are not yet there. I did the examination. Help on'). A 'Send Message' button is located below the message field. At the bottom of the page, there is a copyright notice: '© 2026 Student Management System. All rights reserved.'

Student Management

- [Home](#)
- [Dashboard](#)
- [Login](#)
- [Register](#)

Contact Us

Full Name

Email Address

Subject

Message

© 2026 Student Management System. All rights reserved.

Contact form with name, email, subject, and message fields. The form processes user messages and displays success messages upon submission, demonstrating server-side form handling.

Fig 4: Form Validation Screenshot

The screenshot shows a web browser at the URL `localhost/student-management-system/week4/pages/register.php`. The page title is "Student Management". A navigation menu includes links for Home, Dashboard, Login, and Register. The main heading is "Create an Account". The registration form includes fields for Full Name, Email Address, Password, and Confirm Password. The Password field is highlighted with a red border and a tooltip that says "Please fill in this field." A "Register" button is located below the form. A link "Already have an account? Login here" is provided. The footer contains the copyright notice "© 2026 Student Management System. All rights reserved."

Student Management

- [Home](#)
- [Dashboard](#)
- [Login](#)
- [Register](#)

Create an Account

Full Name

Email Address

Password Please fill in this field.

Confirm Password

Already have an account? [Login here](#)

© 2026 Student Management System. All rights reserved.

The screenshot shows a web browser at the URL `localhost/student-management-system/week4/pages/login.php`. The page title is "Student Management". A navigation menu includes links for Home, Dashboard, Login, and Register. The main heading is "Sign In". The login form includes fields for Email Address and Password. The Password field is highlighted with a red border and a tooltip that says "Please fill in this field." A "Sign In" button is located below the form. A link "Don't have an account? Register here" is provided. The page also includes "Demo Credentials" with an email of `admin@example.com` and a password of `admin123`. The footer contains the copyright notice "© 2026 Student Management System. All rights reserved."

Student Management

- [Home](#)
- [Dashboard](#)
- [Login](#)
- [Register](#)

Sign In

Email Address

Password Please fill in this field.

Don't have an account? [Register here](#)

Demo Credentials:

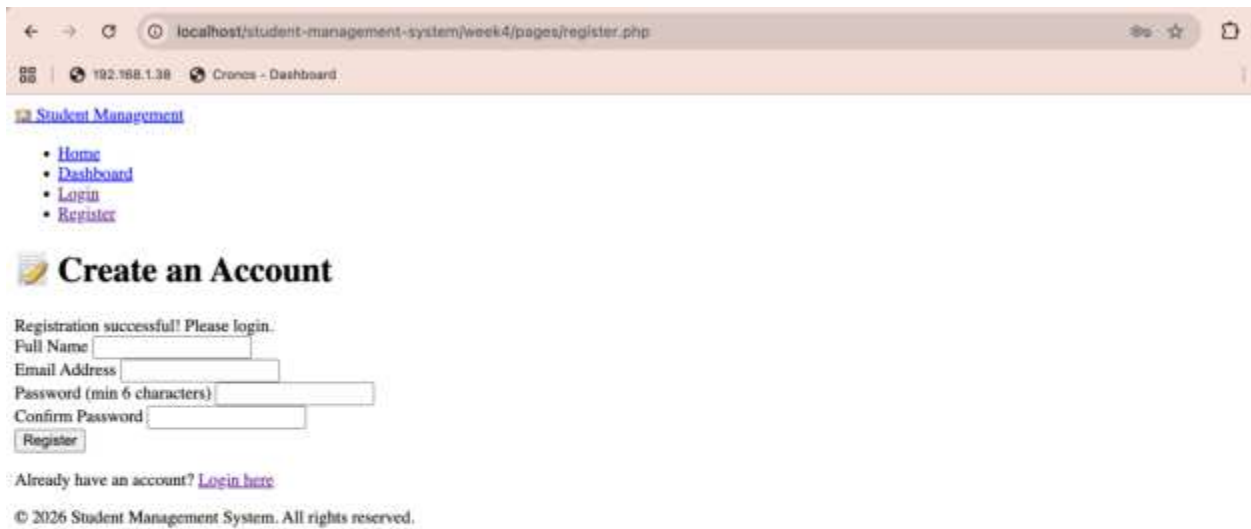
Email: admin@example.com

Password: admin123

© 2026 Student Management System. All rights reserved.

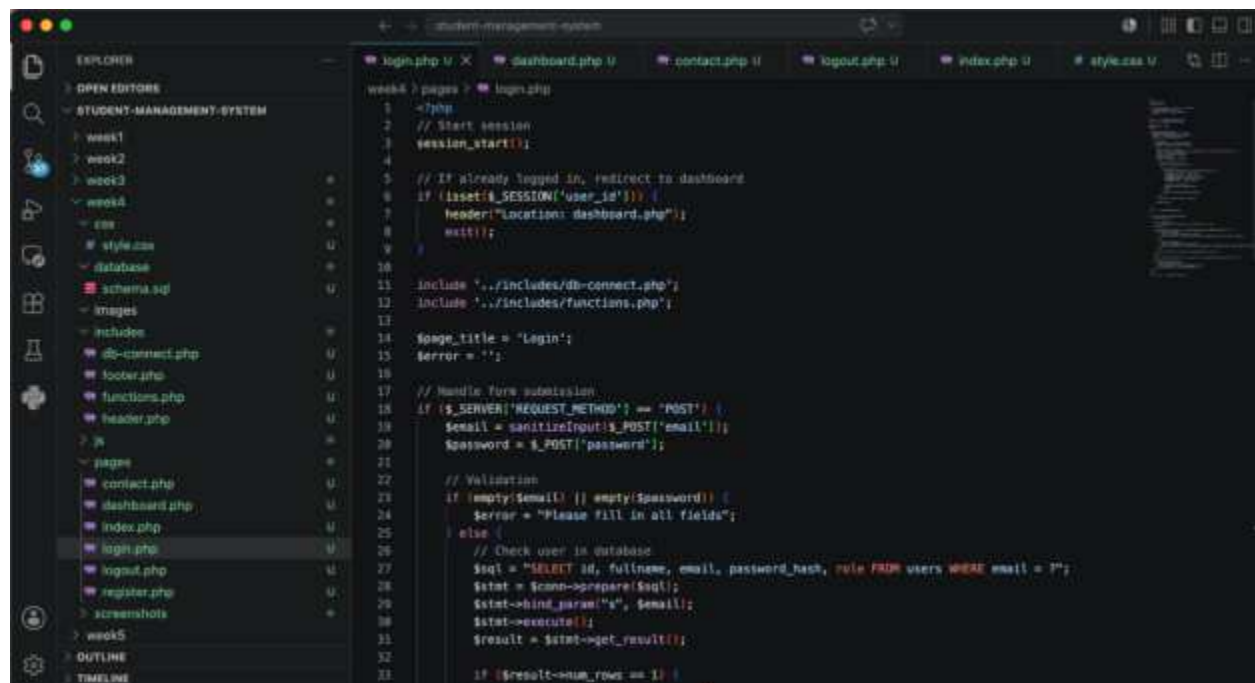
Form validation showing error messages for invalid input. The validation checks for empty fields, email format, and password length, providing immediate feedback to users through the user interface.

Fig 5: PHP Processing Screenshot



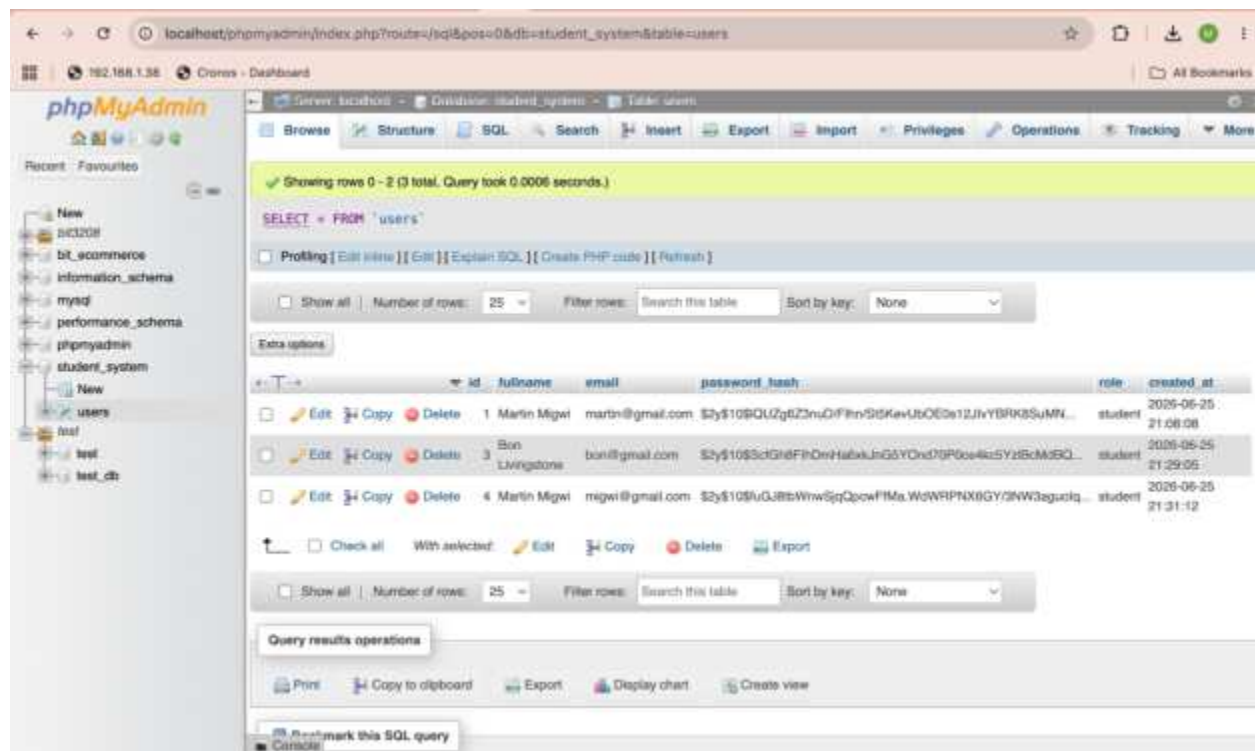
PHP processing page showing successful form submission with data validation and processing confirmation. The page displays the submitted data and a success message, confirming the form was processed correctly.

Fig 6: Folder Structure Screenshot



Professional folder structure showing the organisation of pages, includes, CSS, JS, and database folders. The structure follows best practices for PHP web applications with separate folders for reusable components and page-specific files.

Fig 7: Database Connection Screenshot



Database connection confirmation showing successful connection to the student_system database. The connection script is stored in the includes folder and reused across all pages for efficient code management.

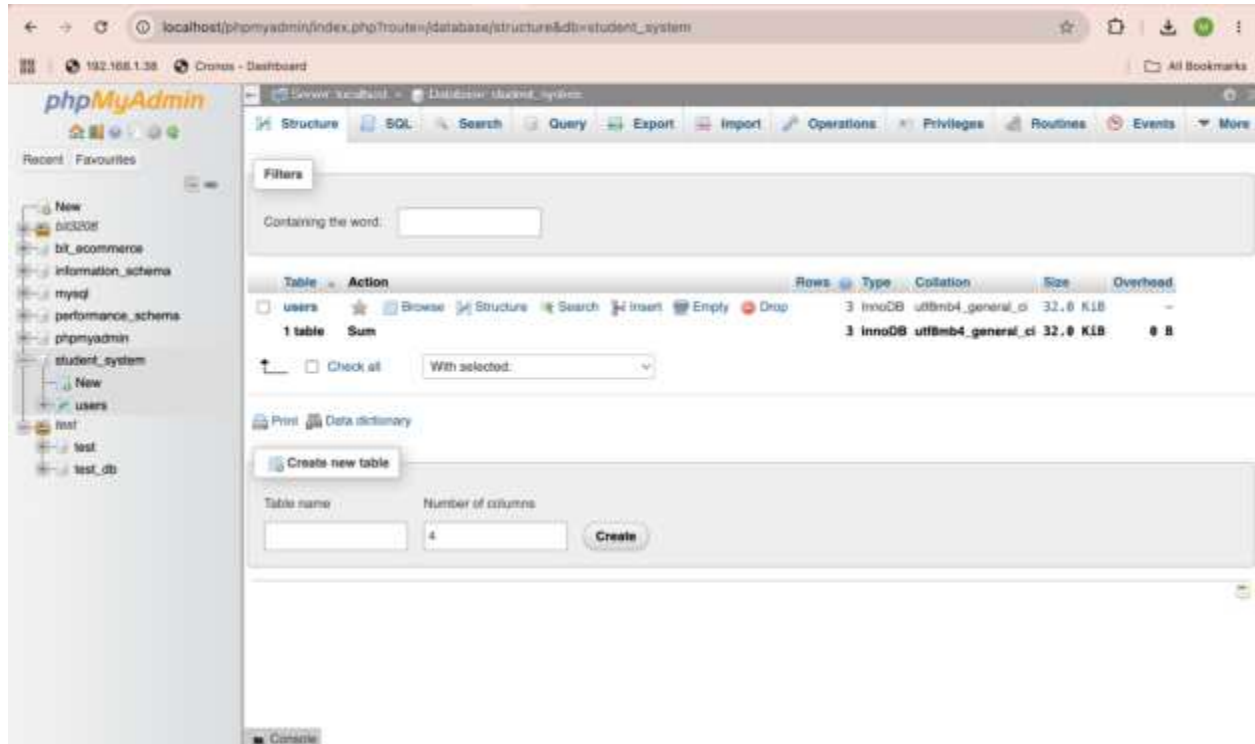
Student Reflection

The folder structure was designed to separate concerns by grouping related files together. The includes folder contains reusable components (header.php, footer.php, db-connect.php, functions.php) while pages contain page-specific logic. This structure improves maintainability and reduces code duplication. The registration form demonstrates secure password handling using password_hash() and prepared statements to prevent SQL injection. The login form uses password_verify() to check credentials before creating sessions. The contact form processes user messages using server-side validation with htmlspecialchars() to prevent XSS attacks. The database connection script is stored in the includes folder and included in all pages using include statements. This approach ensures consistent database connectivity across the application.

WEEK 5: Database and CRUD Operations

Title: Database Creation and CRUD Operations

Fig 1: Database Created Screenshot



phpMyAdmin showing the student_system database with all tables (users, students, courses). The database structure was designed to support the Student Management System with proper relationships between tables.

Fig 2: Tables Created Screenshot

The screenshot shows the phpMyAdmin interface for the 'student_system' database. The 'students' table structure is displayed with the following columns:

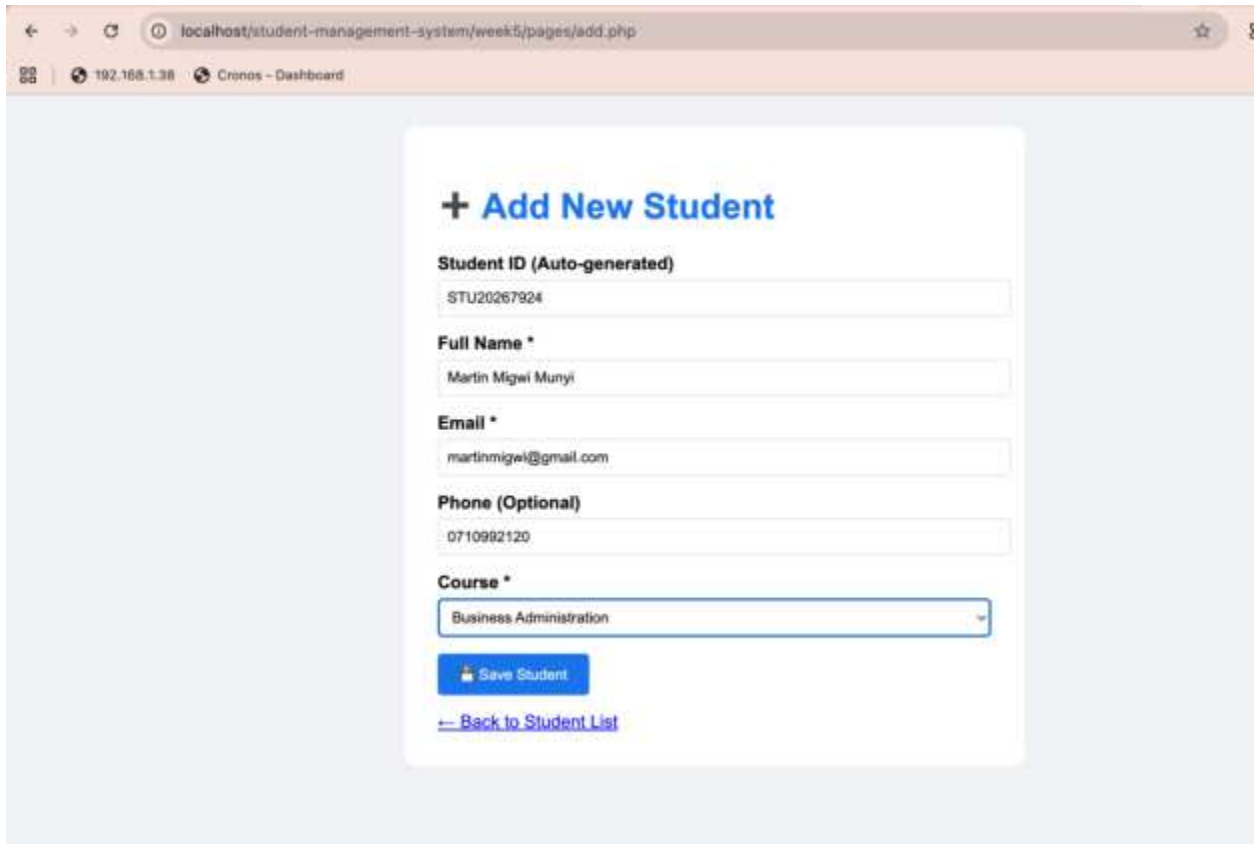
#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	student_id	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More
3	fullname	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
4	email	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
5	phone	varchar(20)	utf8mb4_general_ci		Yes	NULL			Change Drop More
6	course	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
7	created_at	timestamp			No	current_timestamp()			Change Drop More

Below the column list, the 'Indexes' section shows the primary key structure:

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	id	0	A	No	
Edit Rename Drop	student_id	BTREE	Yes	No	student_id	0	A	No	
Edit Rename Drop	email	BTREE	Yes	No	email	0	A	No	

Structure of the students table showing all columns including id, student_id, fullname, email, phone, course, and created_at. The student_id is set as UNIQUE and AUTO_INCREMENT for unique identification.

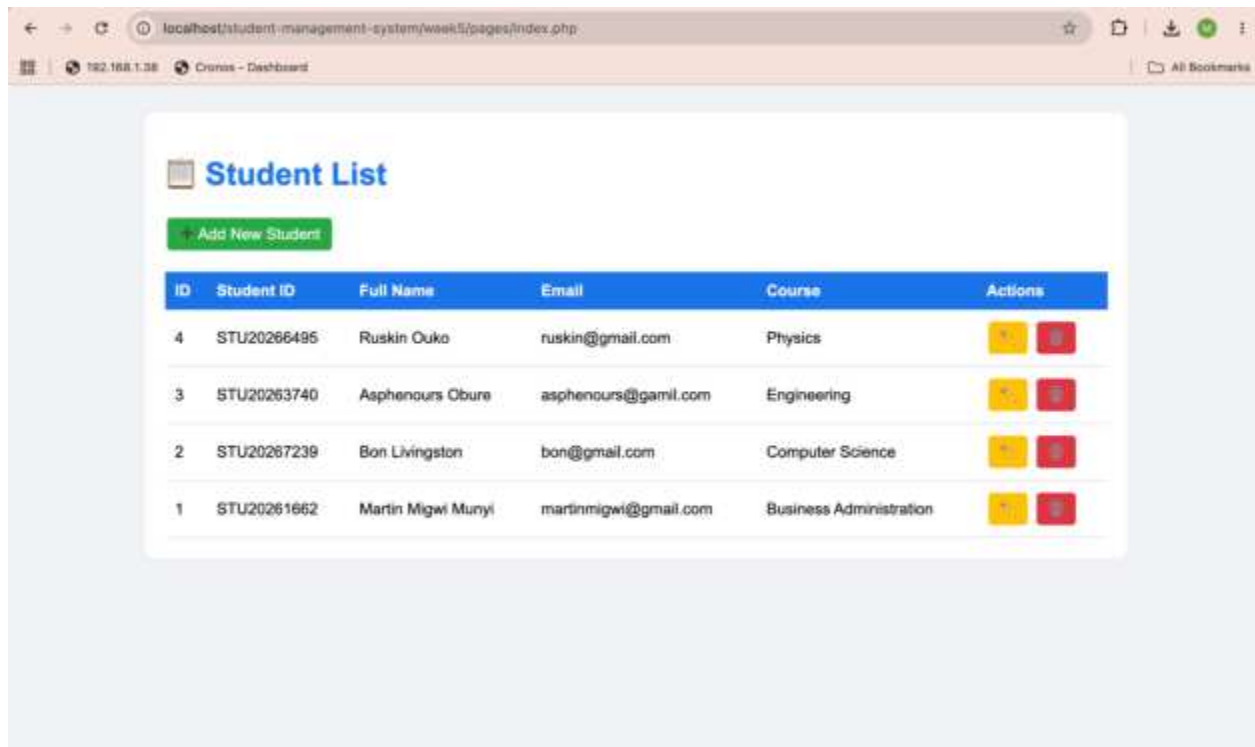
Fig 3: Add Student Screenshot



The screenshot shows a web browser window with the address bar displaying `localhost/student-management-system/week5/pages/add.php`. The browser's address bar also shows the IP address `192.168.1.38` and the page title `Cronos - Dashboard`. The main content area features a white card with a blue header that reads **+ Add New Student**. Below the header, there are several form fields: **Student ID (Auto-generated)** with the value `STU20267924`, **Full Name *** with the value `Martin Migwi Munyi`, **Email *** with the value `martinmigwi@gmail.com`, **Phone (Optional)** with the value `0710992120`, and **Course *** with a dropdown menu showing `Business Administration`. At the bottom of the card, there is a blue button labeled **Save Student** and a link labeled [← Back to Student List](#).

Add student form with auto-generated student ID, name, email, phone, and course fields. The form uses prepared statements to prevent SQL injection and validates input before inserting into the database.

Fig 4: View Students Screenshot



Student list page displaying all student records in a sortable table format. The table includes edit and delete buttons for each record, allowing users to manage student data efficiently.

Fig 5: Edit Student Screenshot

localhost/student-management-system/week5/pages/edit.php?id=4

192.168.1.38 Cronos - Dashboard

Edit Student

✓ Student updated successfully!

Student ID
STU20266495

Full Name *
Ruskin Ouko

Email *
ruskin@gmail.com

Phone
0127381290

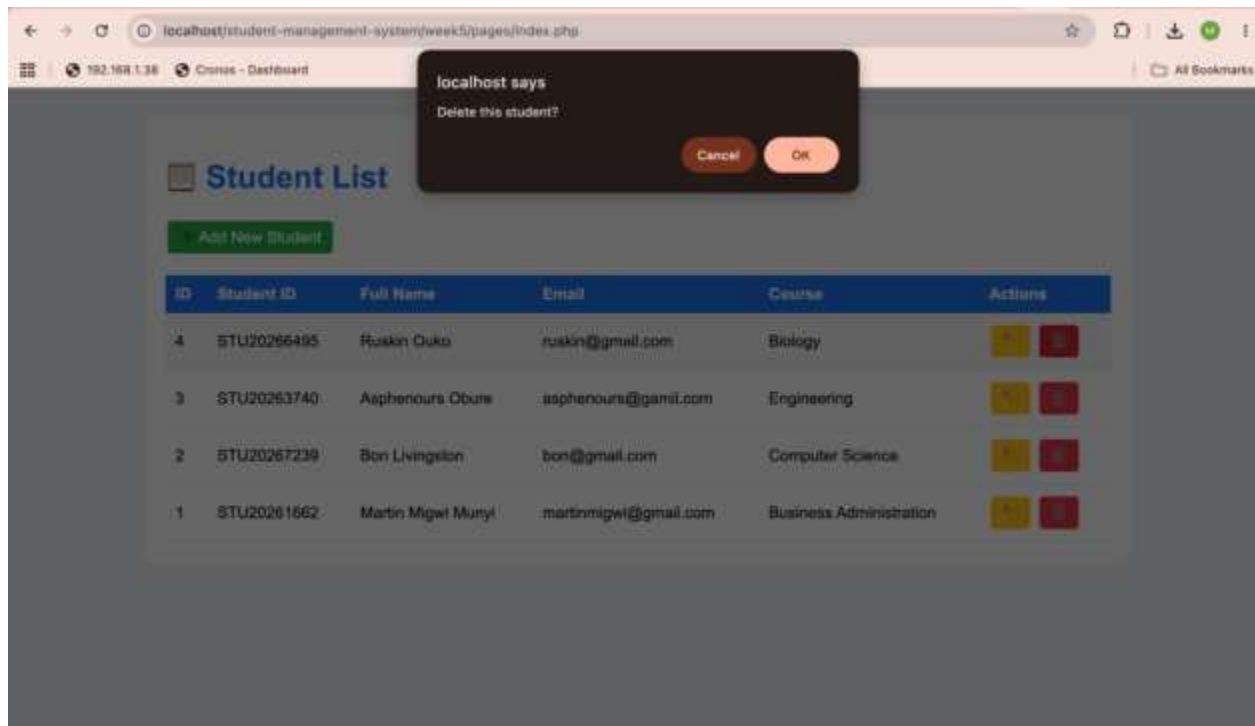
Course *
Biology

 Update Student

[← Back to Student List](#)

Edit student form populated with existing student data. The form validates and updates student records using prepared statements to ensure data security and integrity.

Fig 6: Delete Confirmation Screenshot



Delete confirmation page displaying student details before deletion. The confirmation step prevents accidental data loss by requiring user confirmation before permanently removing records.

Fig 7: Database Connection Screenshot



Database connection confirmation showing successful connection. The script establishes a connection using `mysqli_connect()` and confirms the connection before performing any database operations.

Fig 8: CRUD Complete Screenshot

The figure consists of two screenshots. The top screenshot shows a web application titled "Student List" with a green "Add New Student" button. Below the button is a table with columns: ID, Student ID, Full Name, Email, Course, and Actions. The table contains four rows of student data. The bottom screenshot shows the phpMyAdmin interface for the "student_system" database, displaying the "students" table structure and data. The table has columns: id, student_id, fullname, email, phone, course, and created_at. The data is the same as in the web application screenshot.

Student List

[Add New Student](#)

ID	Student ID	Full Name	Email	Course	Actions
4	STU20266495	Ruskin Ouko	ruskin@gmail.com	Biology	 
3	STU20263740	Asphenours Obure	asphenours@gamil.com	Engineering	 
2	STU20267239	Bon Livingston	bon@gmail.com	Computer Science	 
1	STU20261662	Martin Migwi Munyi	martinmigwi@gmail.com	Business Administration	 

phpMyAdmin

Database: student_system

Table: students

Showing rows 0 - 3 (4 total, Query took 0.0004 seconds)

`SELECT * FROM students`

Extra options

	id	student_id	fullname	email	phone	course	created_at
☐	1	STU20261662	Martin Migwi Munyi	martinmigwi@gmail.com	0710992120	Business Administration	2025-06-25 23:06:43
☐	2	STU20267239	Bon Livingston	bon@gmail.com	0713385063	Computer Science	2025-06-25 23:07:20
☐	3	STU20263740	Asphenours Obure	asphenours@gamil.com	0110624344	Engineering	2025-06-25 23:07:52
☐	4	STU20266495	Ruskin Ouko	ruskin@gmail.com	0127561290	Biology	2025-06-25 23:08:25

Query results operations

[Print](#) [Copy to clipboard](#) [Export](#) [Display chart](#) [Create view](#)

[Bookmark this SQL query](#)

[Console](#)

Student Reflection

The CRUD operations (Create, Read, Update, Delete) form the foundation of the Student Management System. The database was created in phpMyAdmin with three tables: users, students, and courses. The students table uses AUTO_INCREMENT for id and a custom student_id format (STUYYYYYXXXX) for better identification. The Add Student form generates student IDs automatically and uses prepared statements to prevent SQL injection. The View Students page displays records in a table with edit and delete buttons. The Edit page pre-populates the form with existing data and validates updates. The Delete page confirms deletion before removing records. Prepared statements are used throughout all operations to prevent SQL injection attacks. The dashboard provides a quick overview with statistics and recent student activity.

WEEK 6: Database Integration

Title : Advanced Database Integration and Search Functionality

Fig 1: Dashboard Screenshot

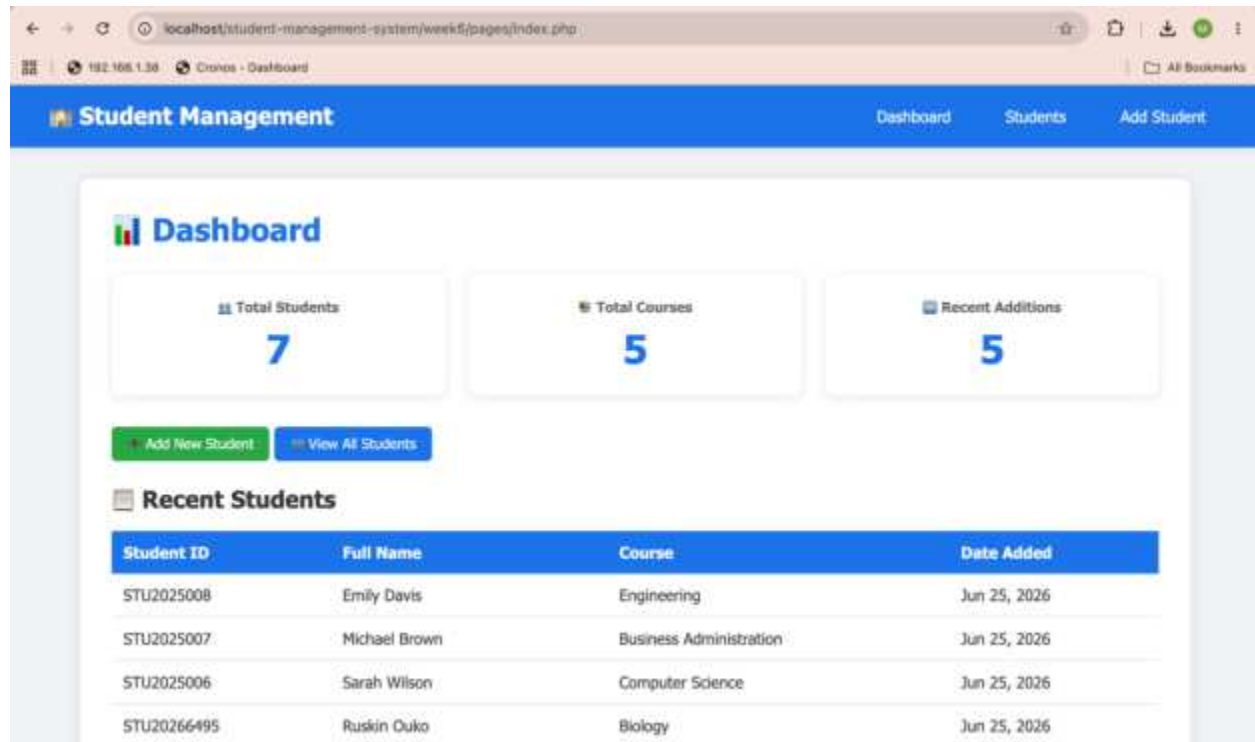
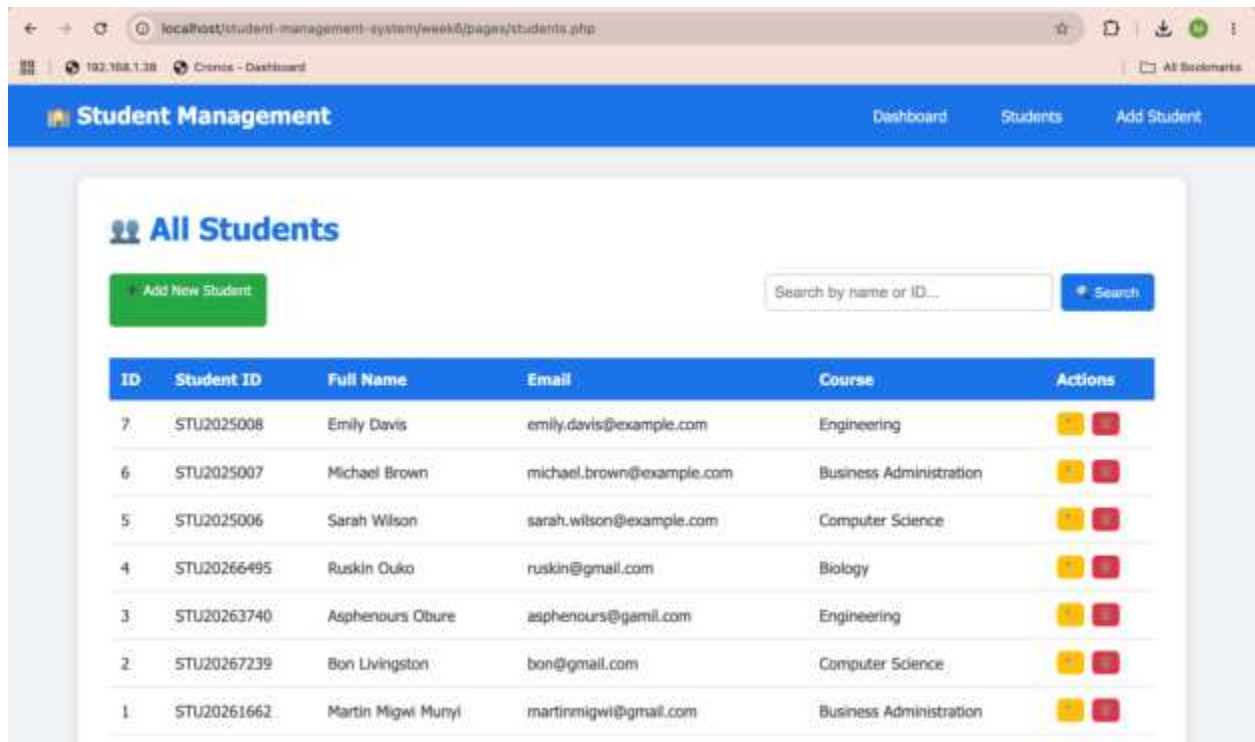


Fig 2: Student List Screenshot



The screenshot displays a web application for student management. The browser address bar shows the URL `localhost/student-management-system/week6/pages/students.php`. The page has a blue header with the title "Student Management" and navigation links for "Dashboard", "Students", and "Add Student". The main content area is titled "All Students" and includes a green "Add New Student" button and a search bar with the placeholder text "Search by name or ID...". Below the search bar is a table listing seven students. Each row in the table includes an "ID" column, a "Student ID", the "Full Name", "Email", "Course", and an "Actions" column with edit and delete icons.

ID	Student ID	Full Name	Email	Course	Actions
7	STU2025008	Emily Davis	emily.davis@example.com	Engineering	 
6	STU2025007	Michael Brown	michael.brown@example.com	Business Administration	 
5	STU2025006	Sarah Wilson	sarah.wilson@example.com	Computer Science	 
4	STU20266495	Ruskin Ouko	ruskin@gmail.com	Biology	 
3	STU20263740	Asphenours Obure	asphenours@gmail.com	Engineering	 
2	STU20267239	Bon Livingston	bon@gmail.com	Computer Science	 
1	STU20261662	Martin Migwi Munyi	martinmigwi@gmail.com	Business Administration	 

Student list with search functionality. Users can search for students by name, student ID, or email. The search results are displayed in the table format with edit and delete actions for each record.

Fig 3: Add Student Screenshot

The screenshot displays a web browser window with the address bar showing `localhost/student-management-system/week5/pages/add-student.php`. The page title is "Student Management" and the navigation bar includes links for "Dashboard", "Students", and "Add Student". The main content area is titled "+ Add New Student". A green success message at the top states: "Student added successfully! ID: STU20267240". Below this, the form contains the following fields:

- Student ID (Auto-generated):** A text input field containing the value "STU20267241".
- Full Name *:** A text input field containing the value "Dancun Mwangi".
- Email *:** A text input field containing the value "dancun@gmail.com".
- Phone:** A text input field containing the value "0710901210".
- Course *:** A dropdown menu with "Data Structures" selected.

At the bottom of the form, there are two buttons: a green "Save Student" button and a blue "Back" button. The browser's status bar at the bottom right shows the URL `localhost/student-management-system/week5/pages/students.php`.

Enhanced add student form with course dropdown populated from the courses table. The form includes validation and error handling with user-friendly success messages.

Fig 4: Edit Student Screenshot

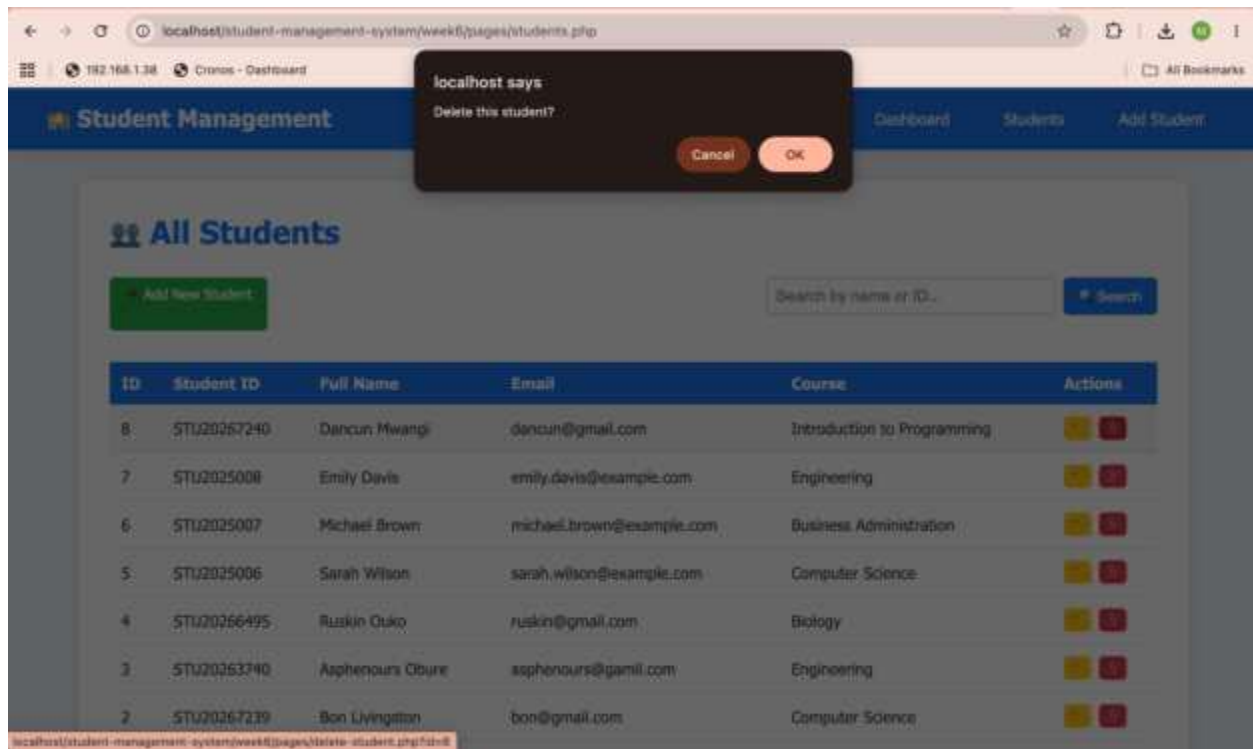
The screenshot shows a web browser window with the URL `localhost/student-management-system/web6/pages/edit-student.php?id=11`. The page has a blue header bar. Below the header, the title "Edit Student" is displayed with a pencil icon. A green success message box states "Student updated successfully!". The form contains the following fields:

- Student ID**: Text input with value "STU20267240".
- Full Name ***: Text input with value "Dancun Mwangi".
- Email ***: Text input with value "dancun@gmail.com".
- Phone**: Text input with value "0710901210".
- Course ***: Dropdown menu with value "Introduction to Programming".

At the bottom of the form are two buttons: "Update Student" (yellow) and "Back" (blue).

Enhanced edit student form with pre-populated data and validation. The form updates student records while maintaining data integrity through prepared statements.

Fig 5: Delete Confirmation Screenshot



Enhanced delete confirmation page with improved styling and error handling. The page confirms deletion before removing records, preventing accidental data loss

Fig 6: Search Functionality Screenshot

The screenshot displays a web application titled "Student Management". The interface includes a navigation bar with links to "Dashboard", "Students", and "Add Student". The main content area is titled "All Students" and features a green "Add New Student" button. A search bar contains the text "@gmail", with "Search" and "Clear" buttons. Below the search bar is a table listing four students. The table has columns for ID, Student ID, Full Name, Email, Course, and Actions. The search results show four students whose email addresses contain "@gmail". At the bottom of the table, it states "Total: 4 students found." The footer of the page reads "© 2026 Student Management System. All rights reserved."

ID	Student ID	Full Name	Email	Course	Actions
8	STU20267240	Dancun Mwangi	dancun@gmail.com	Introduction to Programming	 
4	STU20266495	Ruskin Ouko	ruskin@gmail.com	Biology	 
2	STU20267239	Bon Livingston	bon@gmail.com	Computer Science	 
1	STU20261662	Martin Migwi Munyi	martinmigwi@gmail.com	Business Administration	 

Total: 4 students found.

© 2026 Student Management System. All rights reserved.

Search functionality showing results for a specific search term. The search feature allows users to quickly find students by name or student ID, improving efficiency when managing large datasets.

Fig 7: Database Structure Screenshot

The screenshot displays the phpMyAdmin interface for the 'student_system' database. The 'students' table is selected, and its structure is shown in 'Table structure' view. The table has the following columns:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	student_id	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More
3	fullname	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
4	email	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
5	phone	varchar(20)	utf8mb4_general_ci		Yes	NULL			Change Drop More
6	course	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
7	created_at	timestamp			No	current_timestamp()			Change Drop More

Below the table structure, there are options to 'Check all', 'With selected', 'Browse', 'Change', 'Drop', 'Primary', 'Unique', 'Index', 'Spatial', and 'Fulltext'. There are also buttons for 'Add to central columns' and 'Remove from central columns'.

The 'Indexes' section at the bottom shows the following indexes:

Action	KeyName	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit a) Rename Drop	PRIMARY	BTREE	Yes	No	id	5	A	No	
Edit a) Rename Drop	student_id	BTREE	Yes	No	student_id	5	A	No	
Edit a) Rename Drop	email	BTREE	Yes	No	email	5	A	No	

Complete database structure showing all tables and their relationships. The schema includes users, students, and courses tables with proper primary and foreign key relationships.

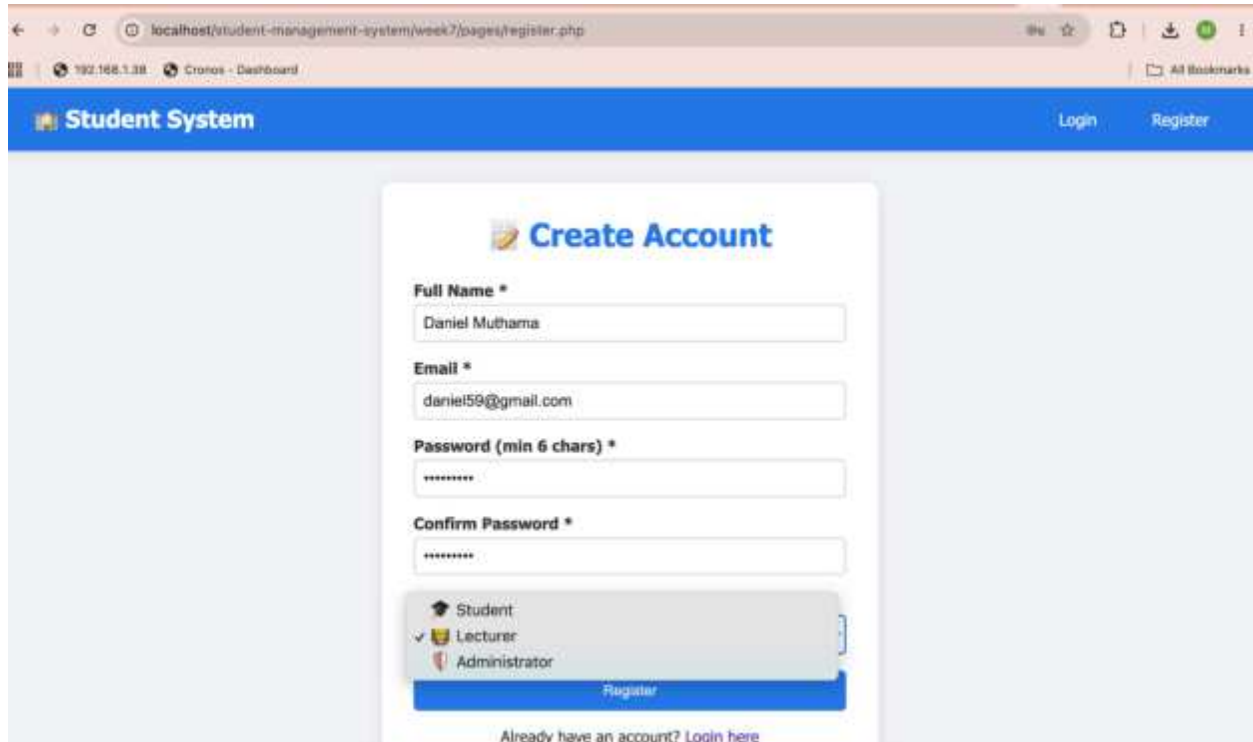
Student Reflection:

Week 6 focused on advanced database integration and improving user experience. The dashboard now displays meaningful statistics including total students, total courses, and recent additions. The search functionality allows users to find students by name, student ID, or email, significantly improving efficiency when managing large datasets. The course dropdown is dynamically populated from the courses table, ensuring data consistency. All database operations use prepared statements to prevent SQL injection and maintain security. The user interface was enhanced with improved styling, user-friendly messages, and better error handling. The database structure was reviewed and refined to support all system features, with proper relationships between users, students, and courses tables.

WEEK 7: User Authentication and Sessions

Title : User Authentication, Sessions, and Role-Based Access Control

Fig 1: Registration Screenshot



The screenshot shows a web browser window with the URL `localhost/student-management-system/week7/pages/register.php`. The page has a blue header with the text "Student System" and links for "Login" and "Register". The main content area features a "Create Account" form. The form includes the following fields:

- Full Name ***: A text input field containing "Daniel Muthama".
- Email ***: A text input field containing "daniel59@gmail.com".
- Password (min 6 chars) ***: A password input field with masked characters "*****".
- Confirm Password ***: A password input field with masked characters "*****".
- Role Selection**: A list of roles with checkboxes:
 - ☐ Student
 - ☒ Lecturer
 - ☐ Administrator

Below the role selection is a blue "Register" button. At the bottom of the form, there is a link: "Already have an account? [Login here](#)".

Registration page with full name, email, password, confirm password, and role selection fields. Password hashing using `password_hash()` ensures secure password storage. The page validates input and displays success/error messages to users.

Fig 2: Login Screenshot

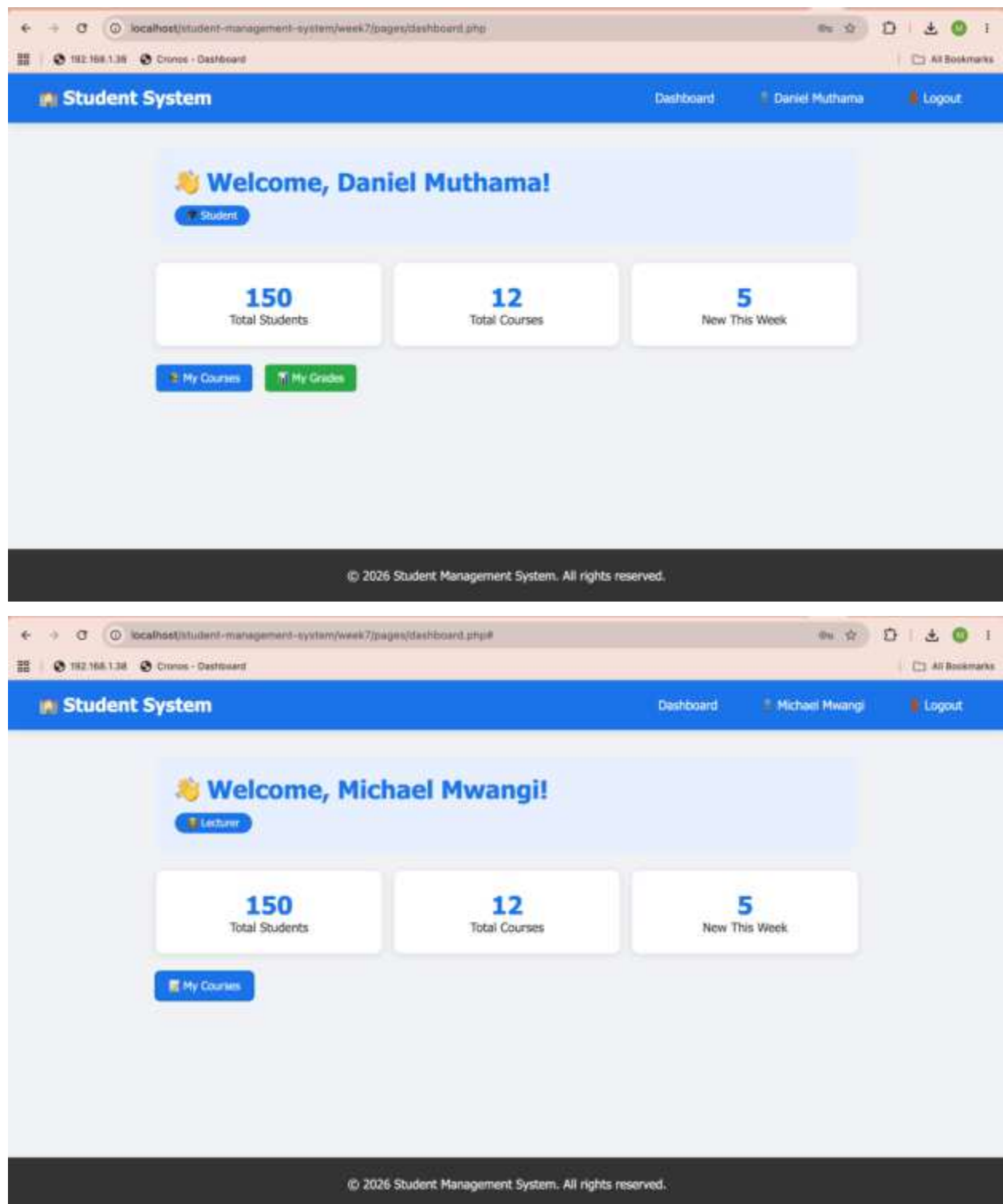
The screenshot displays a web browser window with the URL `localhost/student-management-system/week7/pages/login.php`. The page title is "Student System". The navigation bar includes "Login" and "Register" links. The central "Sign In" form contains the following elements:

- Email:** A text input field with the placeholder "Enter your email".
- Password:** A text input field with the placeholder "Enter your password".
- Sign In:** A blue button.
- Demo Credentials:**
 - Admin:** `admin@example.com / password123`
 - Lecturer:** `lecturer@example.com / password123`
 - Student:** `student@example.com / password123`
- Registration Link:** "Don't have an account? [Register here](#)"

The footer contains the text: "© 2026 Student Management System. All rights reserved."

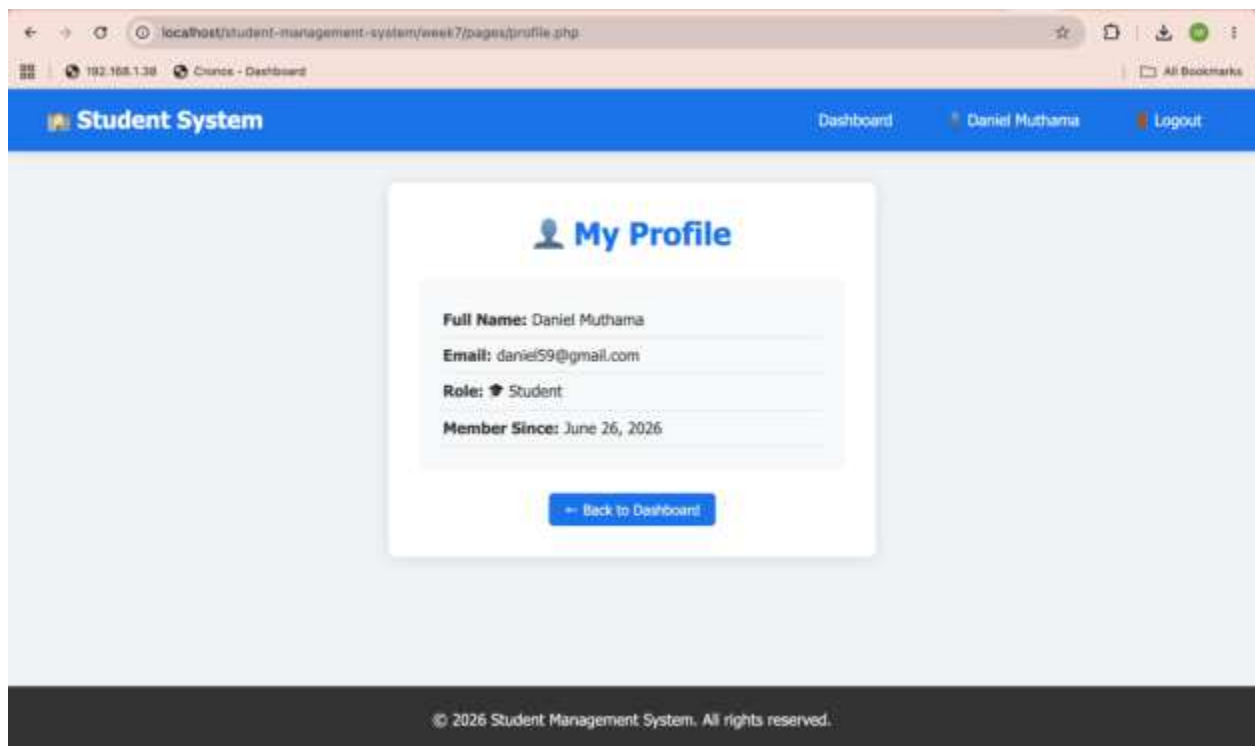
Login page with email and password fields. The page uses `password_verify()` to check credentials and creates sessions upon successful authentication. Demo credentials are displayed for testing purposes.

Fig 3: Dashboard Screenshot



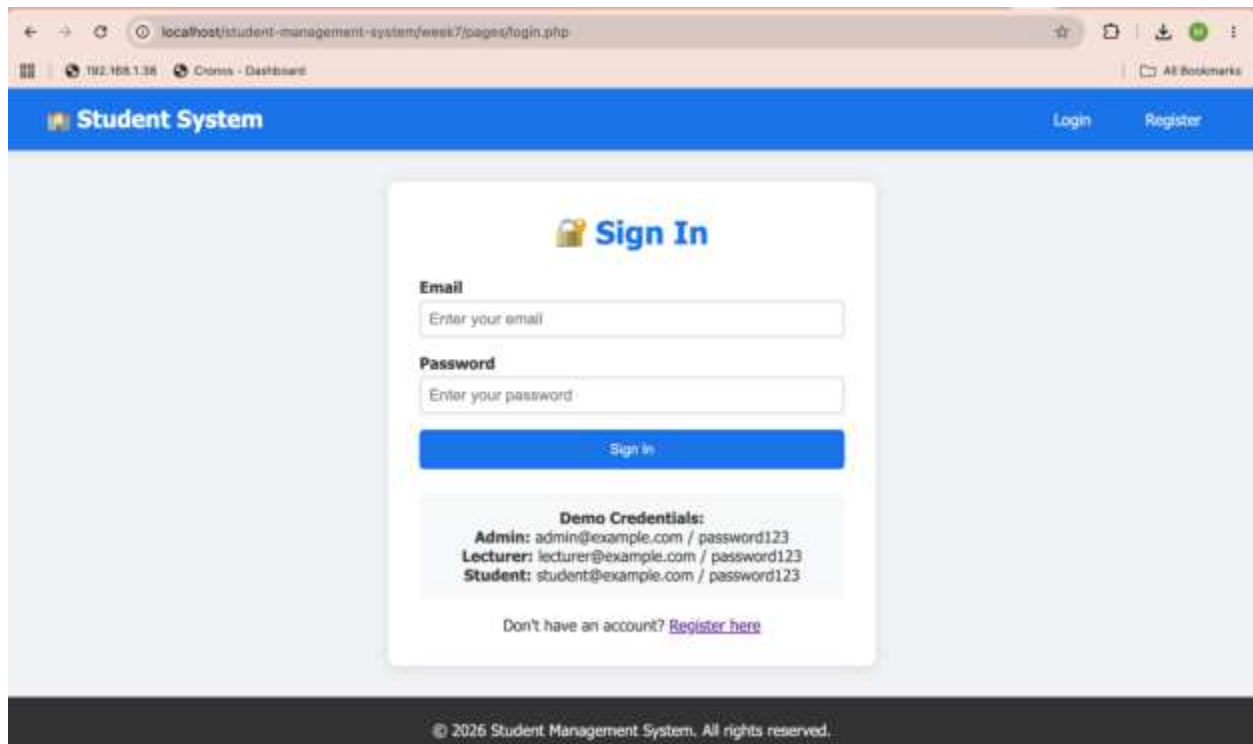
Role-based dashboard showing user-specific content. The dashboard displays the user's role badge, statistics, and action buttons based on permissions (Admin, Lecturer, Student).

Fig 4: Profile Screenshot



User profile page showing account details including full name, email, role, and registration date. The page provides a central location for users to view and manage their account information.

Fig 5: Logout Screenshot



Logout functionality - users are redirected to the login page after session destruction. The logout process uses `session_destroy()` to clear all session data, ensuring secure logout.

Fig 6: Users Table Screenshot

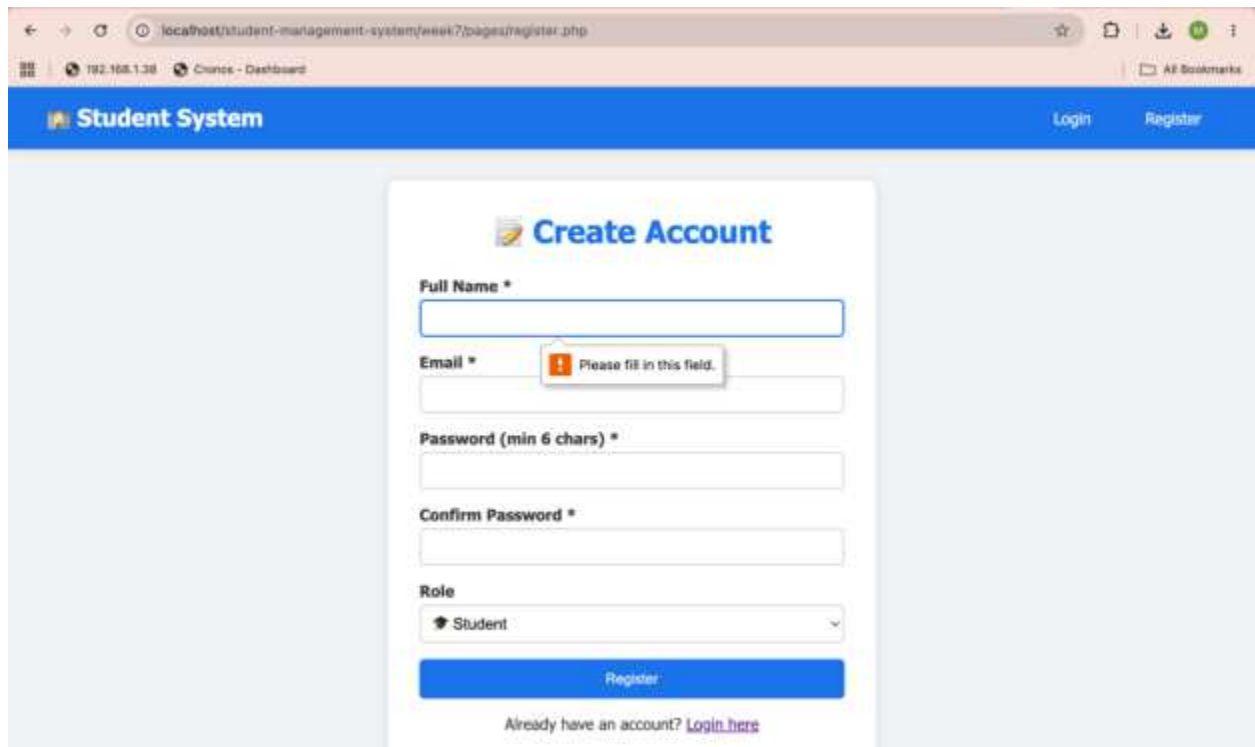
The screenshot shows the phpMyAdmin interface with the 'Users' table structure displayed. The table has six columns: id, fullname, email, password_hash, role, and created_at. The 'id' column is the primary key. The 'password_hash' column is a VARCHAR(255) type, used for storing bcrypt hashed passwords. The 'created_at' column is a timestamp type. The interface also shows the 'Indexes' section with two indexes: 'PRIMARY' on the 'id' column and 'email' on the 'email' column.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	fullname	varchar(100)	utf8mb4_general_ci		Yes	NULL			Change Drop More
3	email	varchar(100)	utf8mb4_general_ci		Yes	NULL			Change Drop More
4	password_hash	varchar(255)	utf8mb4_general_ci		Yes	NULL			Change Drop More
5	role	varchar(50)	utf8mb4_general_ci		Yes	student			Change Drop More
6	created_at	timestamp			No	current_timestamp()			Change Drop More

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	id	5	A	No	
Edit Rename Drop	email	BTREE	Yes	No	email	5	A	Yes	

Users table structure showing all columns including id, fullname, email, password_hash, role, and created_at. The password_hash column stores bcrypt hashed passwords for security.

Fig 7: Protected Page Screenshot



The screenshot shows a web browser window with the URL `localhost/student-management-system/week7/pages/register.php`. The page title is "Student System" and it features a blue header bar with "Login" and "Register" links. The main content area displays a "Create Account" form. The form includes the following fields and elements:

- Full Name ***: A text input field.
- Email ***: A text input field with a red error message icon and the text "Please fill in this field."
- Password (min 6 chars) ***: A text input field.
- Confirm Password ***: A text input field.
- Role**: A dropdown menu with "Student" selected.
- Register**: A blue button.
- Already have an account? [Login here](#)**: A link at the bottom of the form.

Protected page redirection - users without valid sessions are redirected to the login page. Authentication middleware checks for valid sessions before allowing access to protected pages.

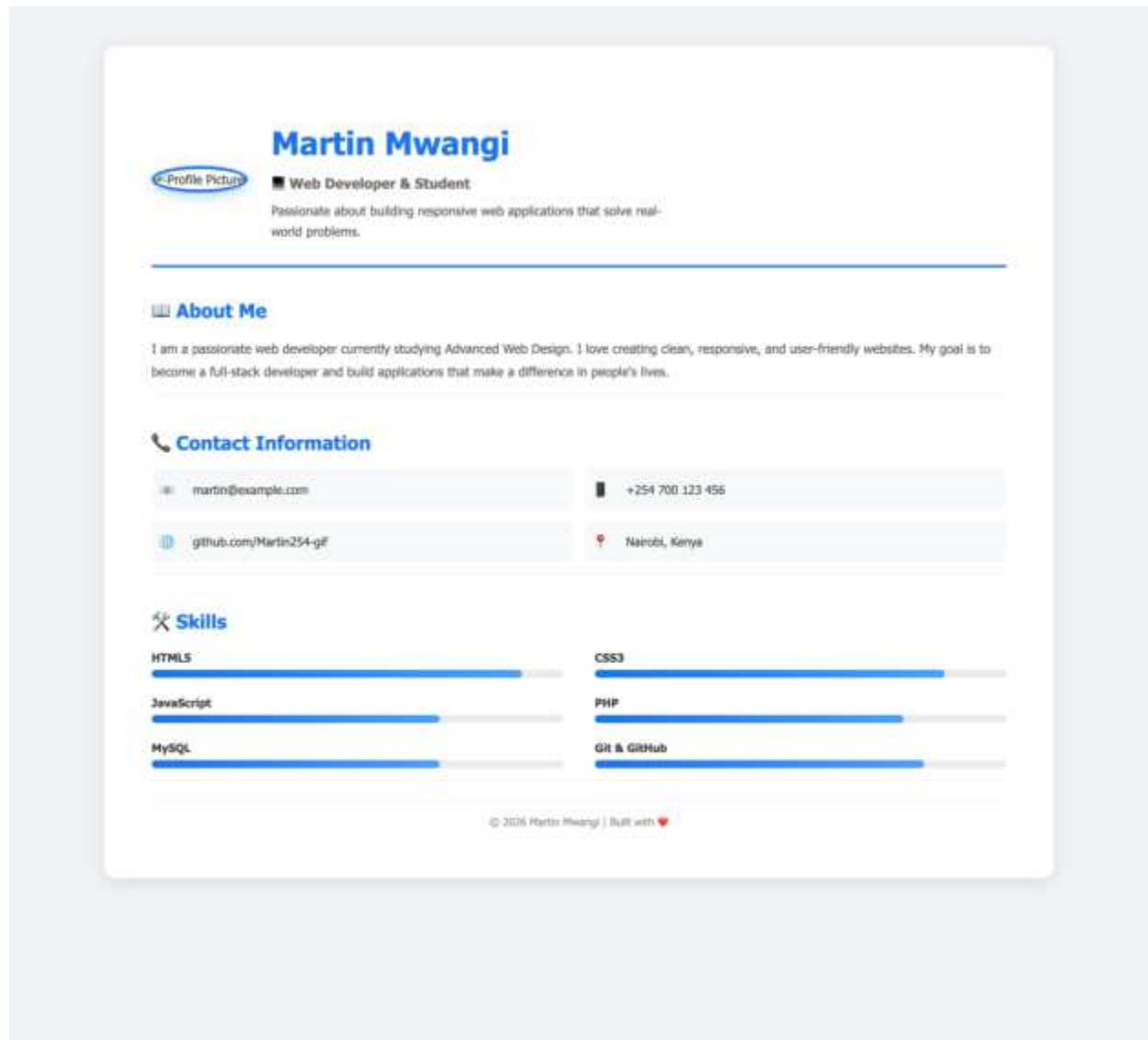
Student Reflection:

Week 7 implemented a complete authentication system with secure login, registration, session management, and role-based access control. User passwords are hashed using `password_hash()` with `bcrypt`, ensuring secure storage. Registration validates input, checks for duplicate emails, and creates new user accounts with assigned roles. Login uses `password_verify()` to check credentials and creates sessions with user data (id, fullname, email, role). Session management protects pages by checking for valid sessions before displaying content. Role-based access control displays different content and actions based on user roles (Admin, Lecturer, Student). Logout uses `session_destroy()` to clear all session data. The authentication system uses prepared statements throughout to prevent SQL injection and ensure security.

WEEK 8: Responsive Web Design

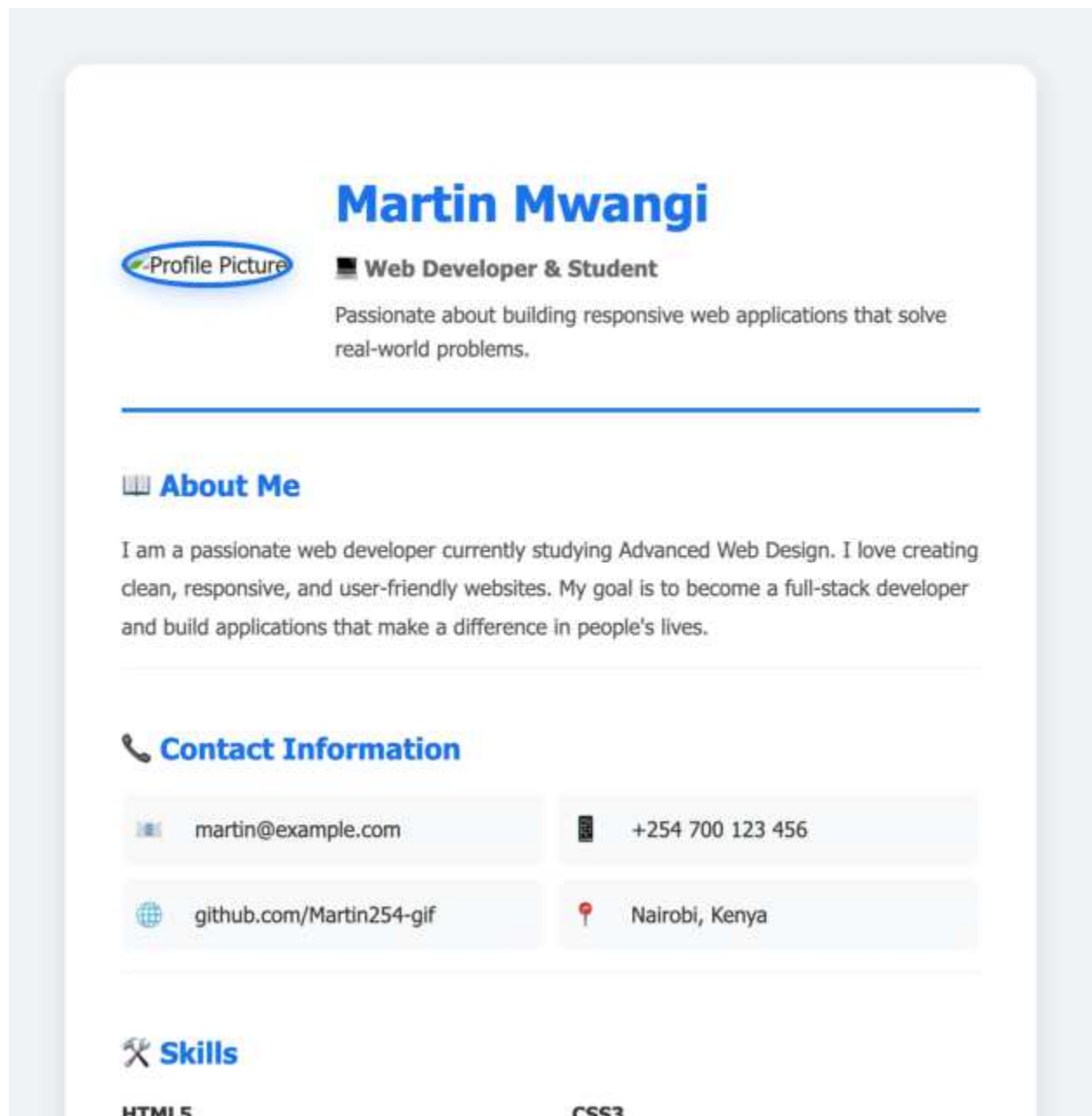
Title : Responsive Web Design and Mobile-First Development

Fig 1: Desktop View Screenshot



Desktop view showing the full layout with multiple columns, sidebar navigation, and content displayed in a grid. The design takes advantage of larger screens to display more information efficiently.

Fig 2: Tablet View Screenshot



Tablet view showing the responsive layout adjusted for medium-sized screens. The interface maintains usability while adapting to the smaller screen dimensions of tablets.

Fig 3: Mobile View Screenshot

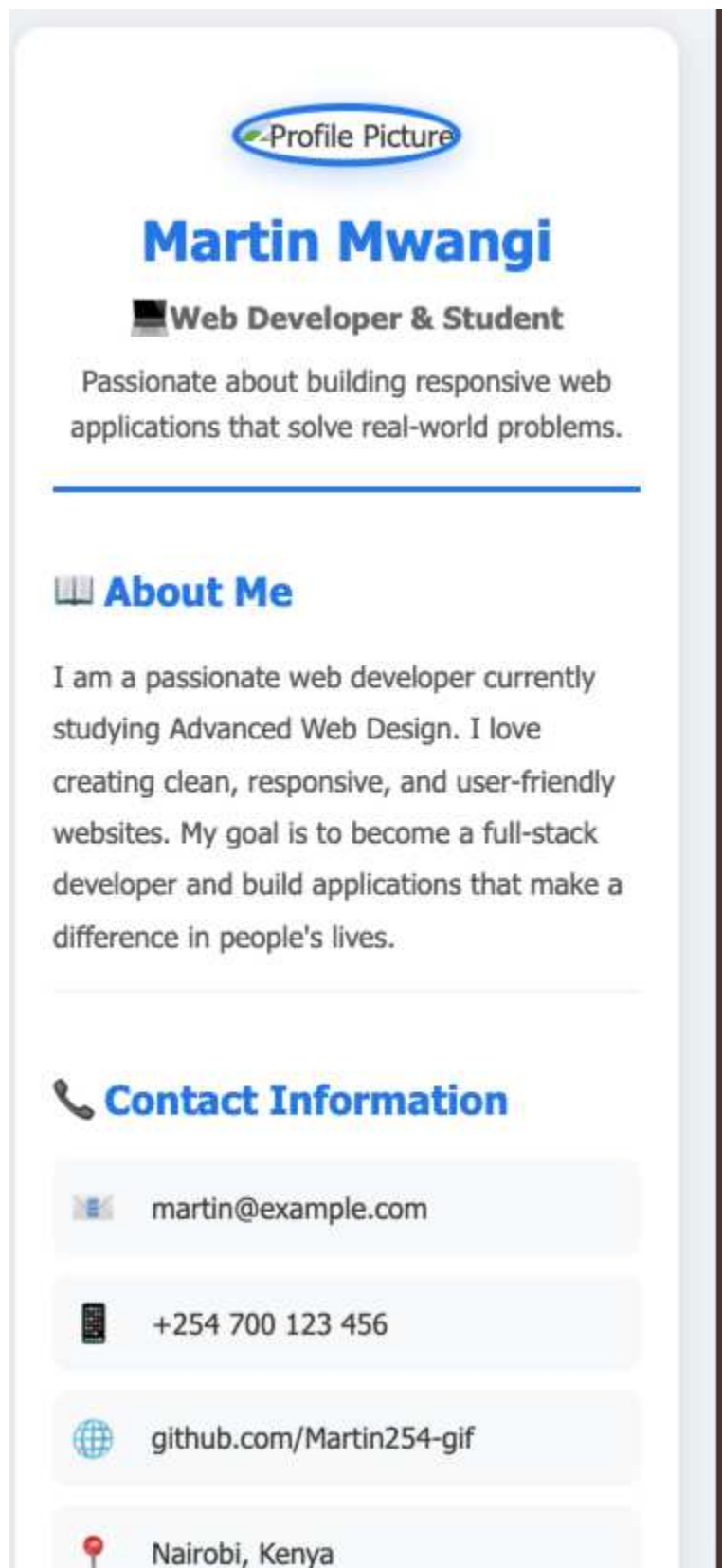


Fig 4: Responsive Navigation Screenshot

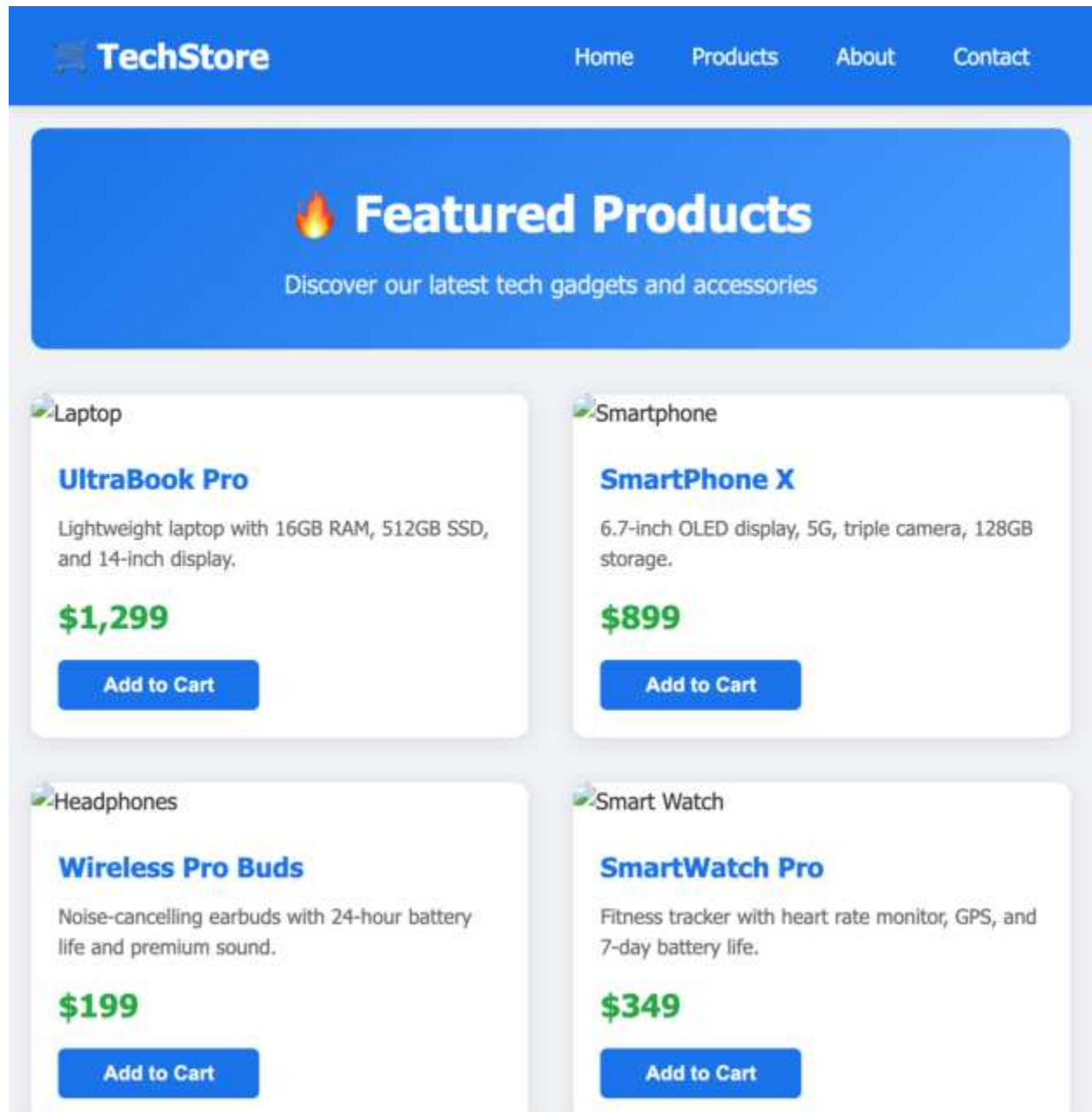


Fig 5: Responsive Table Screenshot

Student Management

DashboardStudentsAdd StudentProfileLogout

Student List

Search students...

Search

ID	Student ID	Full Name	Email	Course	Actions
1	STU2025001	John Doe	john@example.com	Computer Science	
2	STU2025002	Jane Smith	jane@example.com	Business Admin	
3	STU2025003	Bob Wilson	bob@example.com	Engineering	

+ Add New Student

Student ID

STU2025008

Full Name *

Enter full name

Email *

Enter email

Course *

-- Select --

Fig 6: Responsive Form Screenshot

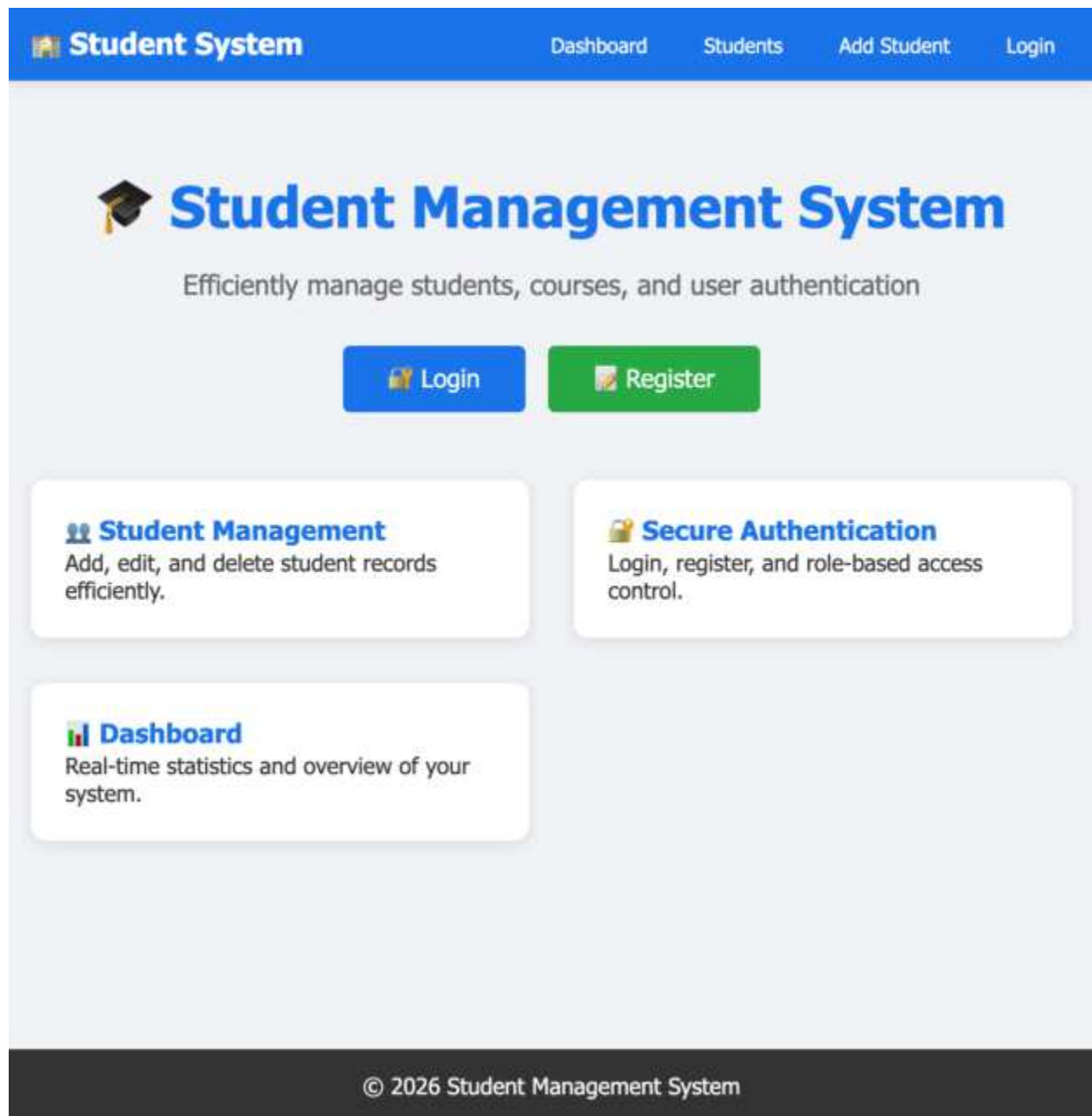


Fig 7: Mobile Menu Screenshot



Student System

Dashboard

Students

Add Student

Login



Student Management System

Efficiently manage students,
courses, and user authentication

Student Reflection

Week 8 focused on responsive web design and mobile-first development principles. The design uses CSS Flexbox for flexible layouts and CSS Grid for the product showcase. Media queries are implemented at three breakpoints: 768px (tablet), 1024px (laptop), and 1440px (desktop). The mobile-first approach ensures optimal performance on smaller screens, with enhancements for larger screens. The responsive table converts to cards on mobile, improving readability. The hamburger menu provides easy navigation on small screens. The design was tested using Chrome DevTools device emulator to ensure compatibility across devices. The profile page demonstrates Flexbox for layout, while the product showcase demonstrates CSS Grid for product listings. The school management system was made fully responsive by applying these principles to all pages.

Challenges:

1. Configuring XAMPP on macOS with proper permissions
2. Implementing secure password hashing and verification
3. Creating responsive tables that convert to cards on mobile
4. Managing sessions and role-based access control
5. Preventing SQL injection with prepared statements
6. Resolving Git push errors and authentication issues

Final Conclusion:

The Student Management System is a fully functional web application that demonstrates proficiency in full-stack web development. The project follows best practices for security, code organisation, and user experience. The system is scalable and can be extended with additional features such as grade management, attendance tracking, and fee management. The 8-week journey has provided valuable hands-on experience with PHP, MySQL, HTML, CSS, JavaScript, and responsive design principles. This project serves as a strong foundation for future web development work and demonstrates the ability to deliver a complete software solution from planning to deployment.